

Web Based Class Scheduling for DMU Burie

Campus

A PROJECT REPORT

Submitted by

STUDENT NAME

STUDENT ID

1. *Dessie Aweke*.....14059134
2. *Shibesh Taddele*.....1404221
3. *Yhalem Ayalu*.....1404472
4. *Eyerusalem Girmachew*.....1403837

in partial fulfillment for the award of the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

Under the guidance of

Mr. Andargie Mekonen (M.Sc)

ADVISOR SIGNATURE



DEPARTMENT OF COMPUTER SCIENCE

BURE CAMPUS

DEBRE MARKOS UNIVERSITY

March/202

Absract

this project aims to develop a Web-Based Class Scheduling System for Debre Markos University (DMU) Bure Campus, designed to automate the management of class schedules for the Computer Science Department. The system addresses the challenges of manual class scheduling, such as time conflicts, resource management, and inefficient allocation of classroom spaces. By leveraging modern web technologies such as HTML, CSS, JavaScript, PHP, and SQL, this project provides an intuitive interface for administrators, instructors, students, and department heads to manage class schedules. The system allows administrators to input building and room information, while department heads can assign courses based on available hours and academic calendars. Instructors and students can easily view the class schedule without needing to sign in, streamlining communication and reducing errors. The automated scheduling process optimizes the allocation of resources and ensures a more efficient academic environment. This project aims to improve the overall scheduling experience for both faculty and students at DMU Bure Campus.

Acknowledgement

We first praise almighty God for granting us the strength and wisdom to develop this system. we would like to express our sincere gratitude to our project advisor, Mr. Andargie Mekonen(M.Sc), for their invaluable guidance, support, and encouragement throughout the course of this project. We are also grateful to the faculty and staff of the Computer Science Department at Debre Markos University, Bure Campus, for providing the resources and environment necessary for completing this work. Special thanks to our friends and colleagues for their continuous support and insightful feedback. Finally, we would like to thank our family for their unwavering support and understanding during our studies.

Table of Contents

Chapter one : Introduction	1
1.1. Introduction	1
1.2. Background of the project	1
1.3. Problem Statement	2
1.4. Objective of the project	2
1.4.1. General Objective	2
1.4.2. Specific Objective	2
1.5. Scope of the project	3
1.6. Limitation of the project	3
1.7. significance of the project	3
1.8. System requirements	4
1.8.1. Hardware requirement	4
1.8.2. Software requirement	4
1.8.3. Programming Language	5
1.9. Data Collection Methodology	5
1.10. Feasibility Study	6
Chapter two : System Analysis	8
2.1. Overview of the existing system	8
- Users of the existing system	9
2.2. System Requirement Specification	10
2.2.1. Functional requirement	10
2.2.2. Non Functional Requirements	11
2.2.3. Business Rules	13
2.2.4 Change Cases	13
2.3. System requirement analysis	14
2.3.1 Actors and use case identification	14
Chapter three: System Design	40
3.1 Design Class Diagram	40
3.2.Database design /Physical data model	42
3.3 User Interface Design	43
3.4. System Architecture (Deployment Diagram)	46
CHAPTER 4 :IMPLEMENTATION	47
4.1 Overview of the programming language used	47
4.2 Algorithms	47

4.3 Sample Codes	49
CHAPTER 5:TESTING	52
5.1 Sample Test	52
CHAPTER 6:Conclusion and Recommendation	54
6.1. Conclusion	54
6.2. Recommendation	55
6.3. Future enhancement	56
References	57

List of Table

Table 1: Use case Identification and Description for create user account	18
Table 2: Use case Identification and Description for register biulding	19
Table 3: Use case Identification and Description for set acadamic date	20
Table 4 Use case Identification and Description for submit courses	21
Table 5: Use case Identification and Description for deactivate user acount	22
Table 6Use case Identification and Description for update department	23
Table 7: Use case Identification and Description for Create Class Schedule	25
Table 8: Use case Identification and Description for Update Class Schedule	26
Table 9: Use case Identification and Description for Delete Accounts	27
Table 10:Use case Identification and Description for send feedback	28
Table 11:Use case Identification and Description for View feedback	29

List of Figure

Figure 1:Use case Diagram	16
Figure 2:Login sequence diagram	30
Figure 3:Register building Login sequence diagram	30
Figure 4:Submit course sequence diagram	31
Figure 5:Create class schedule sequence diagram	31
Figure 6:Set Academic set sequence diagram	32
Figure 7:Register department set sequence diagram	32
Figure 8: Update class shcedule set sequence diagram	33
Figure 9: Activity Diagram for Login	34
Figure 10: Activity Diagram for Register Building Information	35
Figure 11: Activity Diagram for set academic date	35
Figure 12: Activity Diagram for create class schedule	36
Figure 13: Activity Diagram for update class schedule	37
Figure 14: Activity Diagram for send feedback	37
Figure 15: Activity Diagram for Delete account	38
Figure 16: Analysis Class Diagram	39
Figure 17: Design Class Diagram	41
Figure 18: database modeling diagram	42
Figure 19: Home page interface	43
Figure 20: view schedule Interface	44
Figure 21: Admin Interface	45
Figure 22: Deployment Diagram	46

List of Acronyms

- ✓ Admin,,,,,,,,,,,,,,→ Administrator or Scheduler
- ✓ DMU,,,,,,,,,,,,,,→ Debre Markos University
- ✓ Course,,,,,,,,,,,,,,→ Subject
- ✓ Instructor,,,,,,,,,,,,,,→ Teacher
- ✓ MySQL,,,,,,,,,→ Structured Query Language (database management system)
- ✓ PHP,,,,,,,,,,,,,,→ Hypertext Pre-processor (text editor)
- ✓ Scheduler,,,,,,,,,,,,,,→ Registrar Office or Admin
- ✓ UML,,,,,,,,,,,,,,→ Unified Modelling Languages
- ✓ DBMS,,,,,,,,,,,,,,→Database Manegment System
- ✓ JS,,,,,,,,,,,,,,→Java Script
- ✓ UI,,,,,,,,,,,,,,→User Interface

1 Chapter one : Introduction

1.1. Introduction

At Debre Markos University (DMU) Bure Campus, managing time and resources efficiently is essential for the smooth operation of academic activities. One of the most important administrative tasks is class scheduling, which ensures that courses, instructors, and facilities are properly organized. However, the university still relies on manual scheduling methods, which can often be inefficient, prone to mistakes, and time-consuming.

This project, titled "Web-Based Class Scheduling for DMU Bure Campus," aims to solve these challenges by introducing a modern, automated scheduling system. By moving away from the traditional manual approach and adopting a digital solution, the project hopes to improve efficiency, minimize errors, and enhance the overall academic experience at DMU Bure Campus.

The following sections of this report will explore the project in detail, covering its background, objectives, design, implementation, and testing offering a complete picture of the system and its potential impact.

1.2. Background of the project

Class scheduling is one of the most important administrative tasks in academic institutions, related to the effective assignment of courses, instructors, classrooms, and time slots. DMU is a large university located in the northwest of Ethiopia, which currently utilizes a manual scheduling system, reliant on paper documents, spreadsheets, and physical communication between department heads, instructors and administrators .

In this system, department heads gather course information in addition to instructor preferences and create class schedules by hand. These schedules are usually circulated via printed papers or notice boards which is not only cumbersome but time taking too.

1.3. Problem Statement

The class scheduling system at DMU Bure Campus is currently manual, leading to several inefficiencies that disrupt academic operations. Key challenges include:

Frequent Errors: Manual data entry increases the likelihood of mistakes, such as scheduling conflicts and double-booked rooms.

Data Management Issues: Paper-based record-keeping often results in lost, misplaced, or inconsistent information, making future reference difficult.

Slow Information Access: Retrieving essential details like instructor and classroom information takes time due to the lack of a centralized system.

Wasted Time and Resources: Manual scheduling requires significant effort, leading to inefficiency in managing time and manpower.

Implementing an automated class scheduling system will address these challenges by reducing errors, improving data management, and enhancing the efficiency of academic planning at DMU Bure Campus.

1.4. Objective of the project

1.4.1. General Objective

The general objective of this project is to develop Web based class scheduling system for DMU burie campus.

1.4.2. Specific Objective

- ✓ Designing an interactive and user-friendly interface for seamless navigation and data entry.

- ✓ Ensuring real-time updates and accurate scheduling information to support effective decision-making.
- ✓ Storing all scheduling data in a centralized, accessible, and manageable MySQL database.
- ✓ Preventing scheduling conflicts, such as double-booked rooms or overlapping instructor assignments.
- ✓ Providing quick access to room availability, instructor assignments, and course details for users.
- ✓ Securing scheduling data through user authentication and role-based access control.
- ✓ Designing a scalable system to accommodate future additions, such as new courses, departments, or academic periods.

1.5. Scope of the project

This project focuses on the current work flow of class scheduling system of DMU burie campus. Generally the scope of the new proposed system includes:-

- ✓ Class scheduling for DMU burie campus.
- ✓ View class scheduled.
- ✓ The new system is going to perform storing data for:-
- ✓ Assigned instructors of each offered courses.
- ✓ Assigned class rooms.
- ✓ Allocated time (period) of each course.

1.6. Limitation of the project

- ✓ The system is web-based only, meaning there is no dedicated mobile application for Android or iOS users.
- ✓ The system has not been deployed on a live server.

1.7. significance of the project

After implementing this automated system it is very easy to schedule class. This system solves the previous time consuming and tiresome system it saves a lot of time and money, scheduling is very fast and delays can be avoided and it helps the university in reducing costs such as labor and stationary. And it avoids wastage of time in teaching and learning process.

1.8. System requirements

There are system requirements for developed any kind of application. We use different type of software and hardware to develop our project. This requirement is like hardware and software.

1.8.1. Hardware requirement

Desktop computer (laptop) : Provides the computing power to write, test, and develop your project.

Server machine : Hosts your web-based system, enabling access by students, instructors, and administrators over the internet.

External hard drive: Ensures safe storage and backup of project files and data

Internet cable: Provides a reliable internet connection for accessing the server and testing the system online.

Printer: Allows you to print out important reports and schedules during project development or testing.

Flash disk : Helps transfer project files easily between different devices for sharing or backup purposes.

1.8.2. Software requirement

Microsoft office :- To write the documentation part and to prepare power point for presentation

Visual studio code : - to write different codes of the projects. It is important because it support different languages like HTML, PHP, and JavaScript. It also used to run and test the project

Client-side: Compatibility with major web browsers (Chrome, Firefox) and operating systems (Windows).

XAMP Server :- to run and test system application.

Wonder share E-draw Max :- To draw UML diagram

1.8.3. Programming Language

Backend Development:

PHP: For server-side scripting and business logic.

Frontend Development:

HTML: For structuring the web pages.

CSS: For styling and creating an attractive user interface.

JavaScript: For adding interactivity and dynamic functionality.

Database Management:

SQL: For storing, retrieving, and managing data efficiently.

1.9. Data Collection Methodology

The data collection methodology for the web-based class scheduling system for DMU Bure Campus involves gathering accurate and relevant information to understand the current scheduling process, identify challenges, and determine the system requirements. The following methods will be employed:

Interviews: Interviews will be conducted with key stakeholders, including department heads, instructors, and administrative staff. This will help gather detailed insights into the current scheduling practices, challenges faced, and their expectations from the new system.

Observation: Direct observation of the current scheduling process will be conducted to understand how timetables are prepared and managed, identify any inefficiencies, and verify the data gathered from interviews and questionnaires.

Document Analysis: Existing class schedules, departmental reports, and policy documents will be reviewed to understand the rules and constraints governing class assignments, instructor availability, and room allocation.

Combining these methods will ensure comprehensive and accurate data collection, providing a solid foundation for designing a system that meets the needs of all stakeholders at DMU Bure Campus.

1.10. Feasibility Study

The feasibility study for the web-based class scheduling system for DMU Bure Campus evaluates the practicality and potential success of developing and implementing the system. This study considers five key aspects:

1. Technical Feasibility: This assesses whether the existing technical infrastructure at DMU Bure Campus can support the system. It includes evaluating the availability of hardware, network facilities, and the technical skills required to develop and maintain the system.

2. Economic Feasibility: This analyzes the cost-effectiveness of the project by comparing the estimated development and implementation costs against the expected benefits. It includes expenses for software development, hardware procurement, and system maintenance.

3.Operational Feasibility: This evaluates whether the system will effectively solve the current scheduling challenges and meet the needs of students, instructors, and administrators. It also considers user acceptance and the ease of integrating the system into daily operations.

4.Schedule Feasibility: This determines whether the project can be completed within the allocated time frame, considering the project scope, development stages, and the availability of resources.

5.Legal Feasibility: This ensures the system complies with relevant rules, policies, and data protection regulations of the university and the country.

feasibility study helps minimize risks and ensures the project's success by confirming its viability from multiple perspectives.

Chapter two : System Analysis

2.1. Overview of the existing system

1.System Workflow

Right now, the scheduling process at DMU Bure Campus is all done manually. Scheduling classes, assigning instructors, and booking classrooms are all handled on paper with handwritten notes. Whenever there's a change, it requires a lot of back-and-forth communication, which slows everything down and makes the whole process less efficient. There's no way to automatically check if there are any scheduling conflicts, so double-booked rooms and overlapping classes happen quite often.

2.Technology Used

There's no specialized software or digital tools involved in the scheduling process. Everything is done through physical files, and sometimes they use basic office tools like printed spreadsheets. This lack of technology makes it difficult to keep track of everything and manage the data smoothly.

3.Strengths of the System

Even though the system isn't the most efficient, its simplicity is a plus for staff members who are used to working with paper and pen. Since everything is written down, there's no need for technical skills or digital training. However, this simplicity is also the reason why things move so slowly.

4. Limitations and Challenges

Manually entering data leaves room for mistakes, such as scheduling conflicts and missing information. It takes a lot of time to update or find information, which significantly slows down decision-making. Additionally, there is no easy way to check classroom availability or view the latest schedule. Information is often spread across different files, leading to missing, duplicated, or outdated data. Without technology to handle repetitive tasks, staff are forced to complete them manually, making the process both time-consuming and inefficient.

6.Performance and Efficiency

The system is slow because everything has to be done by hand, from entering data to checking schedules. Because of human error, accuracy suffers, and with no centralized system in place, it takes much longer to find the information needed to make decisions

- Users of the existing system

An existing system compromises different players to carry out its job. Among those different actors (players), the most common are -

Student: - view the posted schedule.

Instructor: - can be professor, assistance professor, lecturer or assistance lecturer and is a user who gets any information regarding the schedule.

Registrar personnel: - a user who manages the scheduling process.

Department head: - perform course offering process and then sends the offered courses to the scheduler.

Academic council: - this office uses the system to assign building and rooms for departments.

2.2. System Requirement Specification

2.2.1. Functional requirement

The developing system is expected to provide the following functionalities:

FR-1: The system shall allow admin and department head to log in using a unique username and password.

FR-2: The system shall allow users to view you schedule without schedule

FR-3: The system shall allow the Department Head to create, edit, and delete class schedules.

FR-4: The system shall check for scheduling conflicts (e.g., double-booked rooms or overlapping instructor schedules) before saving.

FR-5: The system shall allow modifications to schedules in case of cancellations or rescheduling needs.

FR-6: The system shall allow Department Heads to assign instructors to courses based on their availability.

FR-7: The system shall allow Admins to add, update, and remove department details, including department name, code, and head.

FR-8: The system shall allow Department Heads to add, update, and delete courses.

FR-9: The system shall allow Department Heads to assign courses to batches for specific academic periods.

FR-10: The system shall allow students and the public to view class schedules without authentication.

FR-11: The system shall allow users and the public to submit feedback on the system or schedules.

FR-12: The system shall allow Admins to manage user accounts, including creation, updates, and deactivation.

FR-13: The system shall allow Admins to manage academic periods, including adding and updating academic years and semesters.

FR-14: The system shall allow Admins to view and mark feedback as read.

2.2.2. Non Functional Requirements

Non-functional requirements describe important aspects of the system that users will notice, but they're not directly tied to what the system does—they focus more on how the system performs. While the system must meet all functional requirements, non-functional requirements ensure the system's quality and efficiency.

Technical Requirements

Technical requirements address the technical aspects needed to complete the project successfully. These include factors like performance, reliability, and availability. The system shall be implemented using PHP and MySQL, ensuring compatibility with standard web servers (e.g., Apache) and supporting concurrent user access with minimal latency.

Authentication

To prevent unauthorized access, the system will include strong security measures. Each user will have their own unique username and password to access the system, with passwords hashed using bcrypt to ensure secure storage.

Performance

The system will use efficient algorithms to handle scheduling tasks quickly and accurately. It's designed to provide response times under 2 seconds for typical operations (e.g., schedule creation, public schedule

viewing) and to process conflict checks for schedules in real-time, ensuring data validity.

Usability

The system will offer a user-friendly interface with simple and intuitive navigation. It's designed so users can easily learn how to interact with the system, with clear form labels, dropdown menus, and error messages to guide users, increasing their efficiency.

Authorization

The system will verify user identity through usernames and passwords. It will also determine which users are authorized to access specific features or perform certain operations based on their role (Admin, Department Head, or Student), restricting access to role-specific functionalities.

Modifiability

Authorized users, particularly Admins, will be able to modify the system when needed, making it flexible and adaptable to future changes, such as adding new departments or updating academic periods, with minimal code changes required.

Reliability

The system is expected to run smoothly without frequent crashes or data corruption. It should consistently meet user expectations by ensuring data integrity through database constraints and regular backups.

Security

The system will have strong security features. When a registered user wants to log in, they'll first select their role and then enter their username and password. This approach, combined with session management and input validation, prevents unauthorized access and keeps the system secure.

2.2.3. Business Rules

The process of identifying business rules is often iterative of this system, where rules begin as general statements of policy of the system.

BR-1: Users of the system must have a valid account with a unique username and password to access role-specific functionalities.

BR-2: The system must validate user information during registration, ensuring completeness and uniqueness of username and email before granting access.

BR-3: The system must require inputs for courses, instructors, rooms, sections, academic periods, and student counts to create class schedules.

BR-4: Each course, instructor, room, and department must have a unique identifier to ensure data integrity.

BR-5: Department Heads must assign instructors from their own department to courses and sections.

BR-6: Class schedules must be prepared and finalized before the start of each semester.

BR-7: Only Admins can create, update, or deactivate user accounts and manage department details.

BR-8: Feedback submitted by users or the public must be stored and made available for Admin review.

BR-9: Academic periods (year and semester) must be defined before scheduling can occur.

BR-10: Public schedule viewing must not require user authentication.

2.2.4 Change Cases

Change cases are used to new potential requirements for a system or modification to existing requirements change cases are modelled in a

simple manner. We describe the potential change to our existing requirements to indicate the like lines of that change occurring and changing the potential impacts of that change cases for the proposed project. When new buildings are built in the university our project contains add new building info and when buildings are non-functional our project contains delete buildings operation. generally this project considers future change in the Debre markos university bure campus.

2.3. System requirement analysis

2.3.1 Actors and use case identification

The Web-Based Class Scheduling System for DMU Bure Campus brings together four important actors.

1. Admin : Manages the system's core data and provides essential information for scheduling.

Actor role:-

- ✓ log in the system
- ✓ Register building information (building number, floor number, room name).
- ✓ Set academic dates (academic year, semester start and end dates).
- ✓ manage user accounts.
- ✓ View feedback from student and instructors.

2. Department Head : Creates and manages class schedules for their department.

Actor role:-

- ✓ Login in to the system
- ✓ Gather building information, academic dates, and remaining contact hours from the admin.

- ✓ Submit course information (course code, course title, lecture hours, credit hours).
- ✓ Assign instructors to courses.
- ✓ Create and modify class schedules
- ✓ Add Department Information (includes department name, department code, number of students, section)
- ✓ View feedback related to schedules or system issues.

3. Users: Views class schedules and provides feedback.

Actor role:-

- ✓ View class schedules (course, instructor, room, day, and time).
- ✓ Send feedback on scheduling or academic concerns.

2.3.1.1. Use case Diagram

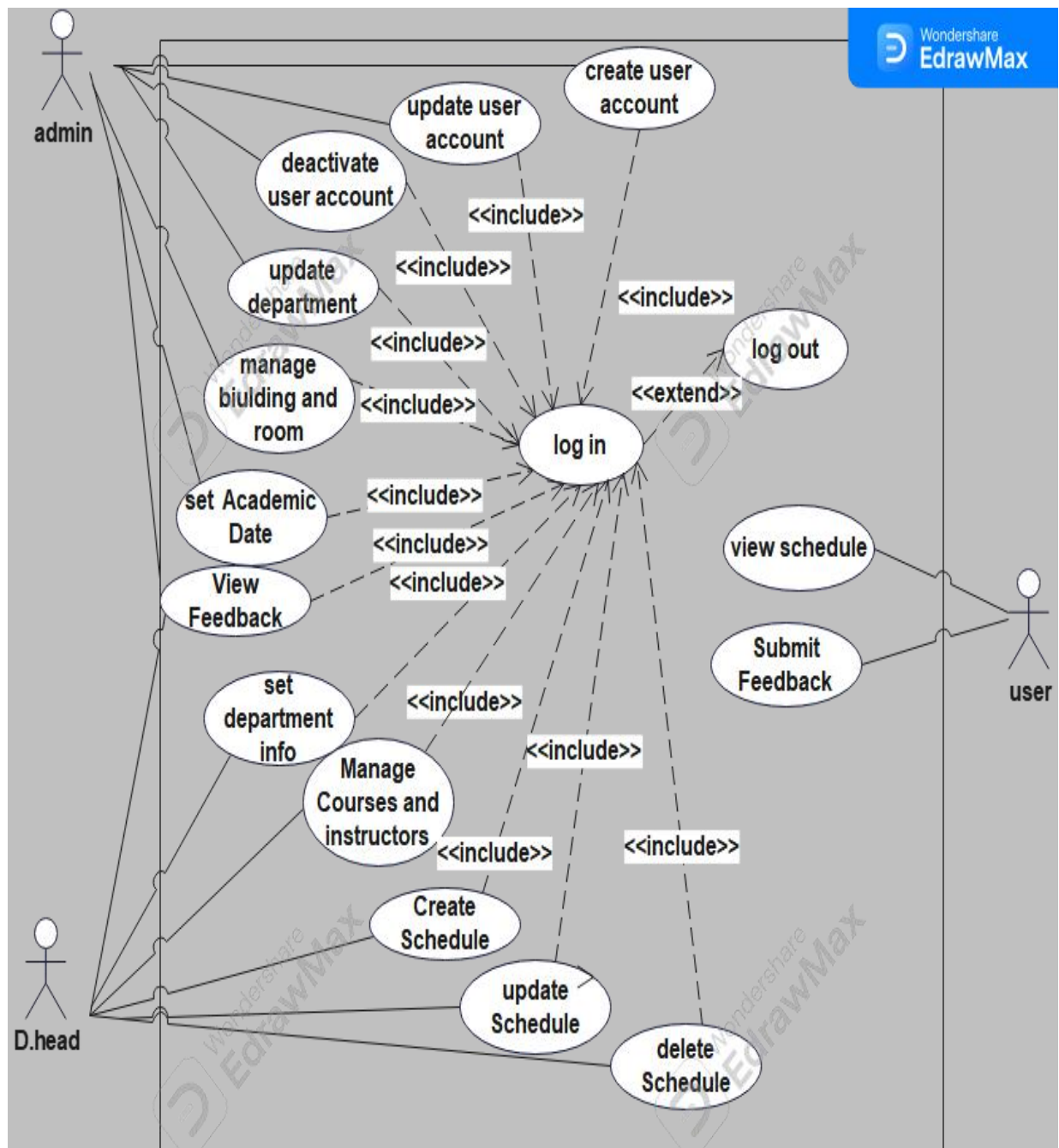


Figure 1::Use case Diagram

Use case Identification and Description

Use case id	UI-01
Usecase name	Create User Account
Actor	Admin
Description	Allows the Admin to create a new user account by entering details such as username, password, email, full name, role, and department. The system validates the input for uniqueness and completeness, creates the account if valid, and shows an error message if invalid.
Precondition	The Admin must be signed in and have access to the user management interface. The database must be accessible.
Postcondition	A new user account is created in the system, and the Admin is notified of the successful creation.
Basic flow	Actor action
	The Admin clicks the "Manage Users" link. 2.The Admin enters the user's username, password, email, full name, role, and department. 3. The Admin clicks the "Create User" button.
Alternative flow	1.If the username or email already exists, the system displays an error message: "Username or email already in use. Please choose another." The form is reloaded for the Admin to retry.

	2.. If the Admin clicks "Create User" without filling all required fields, the system displays an error message: "Please fill in all required fields." The form is reloaded for retry.
--	--

Table 1: Use case Identification and Description for create user account

Use case id	UI-02	
Usecase name	Register Building	
Actor	Admin	
Description	Allows the Admin to register building information, including building number, floor number, and room name. This information is essential for class scheduling.	
Precondition	The Admin must be signed into the system.	
Postcondition	Building information is successfully saved in the system's database.	
Basic flow	Actor action	System response
	1. The Admin navigates to the "Register Building" page.. 3. The Admin enters building number, floor number, and room name.	2. The system displays the building registration form. 5. The system validates the input data. 7.The use case ends.

	4. The Admin clicks the "Save" button.	
Alternative flow	If any required field is empty or invalid, the system displays an appropriate error message and prompts the Admin to correct the input.	

Table 2: Use case Identification and Description for register building

Use case id	UI-03	
Usecase name	Set academic date	
Actor	Admin	
Description	Allows the Admin to set and manage academic dates, including the academic year, semester start date, and end date. This data is essential for creating accurate class schedules.	
Precondition	The Admin must be signed into the system.	
Postcondition	Academic date information is successfully saved in the system's database.	
Basic flow	Actor action	System response
	1. The Admin navigates to the "Set Academic Date" page.. 3. The Admin enters the academic year, semester start date,	The system displays the academic date form. 5.The system validates the input data. 6. If valid, the system saves the academic date information

	and end date. 4. The Admin clicks the "Save" button.	and displays a success message 7.The use case ends.
Alternative flow	If any required field is empty or has an invalid format, the system displays an error message and prompts the Admin to fix the input.	

Table 3: Use case Identification and Description for set academic date

Use case id	UI-04	
Usecase name	Submit Course	
Actor	Department head	
Description	Allows the Department Head to submit course information, including course code, course title, lecture hours and credit hours.	
Precondition	The Department Head must be signed into the system.	
Postcondition	Course information is successfully stored in the system's database and becomes available for scheduling.	
Basic flow	Actor action	System response
	1.The Department Head navigates to the "Submit Course" page. 3.The Department Head enters course	2. The system displays the course submission form. 5. The system validates the input data.

	code, course title, lecture hours, credit hours and remaining contact hours per week. 4. The Department Head clicks the "Submit" button.	6. If valid, the system saves the course information and displays a success message. 7.The use case ends.
Alternative flow	If any required field is empty or contains incorrect data, the system shows an appropriate error message and prompts the Department Head to correct the input.	

Table 4 Use case Identification and Description for submit courses

Use case id	UI-05
Usecase name	Deactivate User Account
Actor	Admin
Description	Allows the Admin to deactivate an existing user account by selecting the user and confirming the action. The system updates the user's status to inactive, preventing login, and notifies the Admin of the outcome. If the user is already inactive or invalid, an error message is shown..
Precondition	The Admin must be signed in and have access to the user management interface. The user account to be deactivated must exist in the system.
Postcondition	The user account is marked as inactive in the system,

	and the Admin is notified of the successful deactivation	
Basic flow	Actor action	
	1.The Admin clicks the "Manage Users" link. 2.The Admin selects a user account to deactivate. 3.The Admin clicks the "Deactivate User" button.	
Alternative flow	1. If the selected user account is already inactive, the system displays an error message: "User account is already deactivated." The user management interface is reloaded. 2. If the Admin attempts to deactivate without selecting a user, the system displays an error message: "Please select a user to deactivate." The interface is reloaded for retry.	

Table 5: Use case Identification and Description for deactivate user account

Use case id	UI-01
Usecase name	Update Department
Actor	Admin
Description	Allows the Admin to update an existing department's details, such as department name, code, or head, by selecting the department and modifying its information. The system validates the input for uniqueness and completeness, updates the department

	if valid, and shows an error message if invalid.
Precondition	The Admin must be signed in and have access to the department management interface. The department to be updated must exist in the system.
Postcondition	The department's details are updated in the system, and the Admin is notified of the successful update.
Basic flow	Actor action
	1.The Admin clicks the "Manage Departments" link. 2.The Admin selects a department to update. 3.The Admin modifies the department's name, code, or head. 4.The Admin clicks the "Update Department" button.
Alternative flow	1.If the updated department code already exists, the system displays an error message: "Department code already in use. Please choose another." The form is reloaded with the entered data for retry. 2. If the Admin submits the form with missing or invalid fields, the system displays an error message: "Please fill in all required fields correctly." The form is reloaded with the entered data for retry.

Table 6 Use case Identification and Description for update department

Use case id	UI-07	
Usecase name	Create Class Schedule	
Actor	Department head	
Description	Allows the Department Head to create a class schedule by using course information, assigned instructors, building information, and academic dates.	
Precondition	The Department Head must be signed into the system, and the following data must already be available: building information, academic dates, submitted courses, and assigned instructors.	
Postcondition	A complete class schedule is saved in the system's database and made accessible to students and instructors.	
Basic flow	Actor action	System response
	The Department Head navigates to the "Create Class Schedule" page. 3. The Department Head selects a course and assigns a room and time slot. 4. The Department Head repeats the process for all	The system displays available courses, assigned instructors, rooms, and time slots. 6.The system checks for conflicts in time, room, and instructor availability. 7.If no conflicts are found, the system saves the schedule and displays a success message.

	courses. 5. The Department Head clicks the "Save Schedule" button.	7.The use case ends.
Alternative flow	If a room or instructor is double-booked, the system displays an error message and highlights the conflict. The Department Head must resolve the issue before proceeding.	

Table 7: Use case Identification and Description for Create Class Schedule

Use case id	UI-08	
Usecase name	Update class schedule	
Actor	Department head	
Description	Allows the Department Head to update an existing class schedule by modifying course assignments, instructor allocations, or room assignments. This ensures the schedule stays up-to-date with any changes.	
Precondition	The Department Head must be signed into the system, and the class schedule must already exist.	
Postcondition	The class schedule is updated with the new information and stored in the system.	
Basic flow	Actor action	System response
	The Department Head navigates to the "Update	2,The system displays the

	Class Schedule" page. 3.The Department Head selects the course or schedule entry to update. 5. The Department Head updates the necessary information. 6. The Department Head clicks the "Save" button.	current class schedule. 4.The system allows the Department Head to modify details like course assignment, instructor, or room. 7. The system validates the updated information. 8. If valid, the system saves the updated class schedule and displays a success message. 9.The use case ends.
Alternative flow	If the Department Head fails to enter valid data, the system displays an error message and prompts for corrections.	

Table 8: Use case Identification and Description for Update Class Schedule

Use case id	UI-11
Usecase name	Delete user account
Actor	admin
Description	Allows the Admin to delete user accounts from the system. This feature is used to remove inactive or unauthorized users.
Precondition	The Admin must be signed into the system.
Postcondition	The selected user account is successfully removed

	from the system	
Basic flow	Actor action	System response
	The Admin navigates to the "Manage Users" page. The Admin selects the user account to delete. 5. The Admin confirms the deletion.	The system displays a list of registered user accounts. 4.The system displays a confirmation prompt. 6. The system removes the user from the database and shows a success message. 7. The use case ends.
Alternative flow	If the Admin tries to delete a non-existent or protected account (e.g., another Admin), the system displays an error message and prevents the deletion.	

Table 9: Use case Identification and Description for Delete Accounts

Use case id	UI-12
Usecase name	Send Feedback
Actor	user
Description	Allows instructors and students to send feedback regarding class schedules, system issues, or academic concerns.Feedback is saved in the system and accessible by the Admin or Department Head for review and resolution.
Precondition	The user (Instructor or Student) must access the system

	and view the class schedule or system features.	
Postcondition	Feedback is successfully saved in the system and made available for review by the Admin or Department Head.	
Basic flow	Actor action	System response
	The user navigates to the "Send Feedback" page. 3. The user fills out the feedback form and clicks the "Submit" button.	The system displays a feedback form (text area, subject, optional attachments). The system validates the input and saves the feedback. 5. The system displays a success message. 6. The use case ends.
Alternative flow	If the user submits the form without filling required fields, the system shows an error message and prompts for completion.	

Table 10: Use case Identification and Description for send feedback

Use case id	UI-13
Usecase name	View feedback
Actor	Admin, Department Head
Description	Allows the Admin and Department Head to view the feedback submitted by students and instructors regarding

	class schedules or system issues.	
Precondition	The Admin or Department Head must be logged in to the system. Feedback must be submitted by students or instructors.	
Postcondition	The Admin or Department Head successfully views the submitted feedback.	
Basic flow	Actor action	System response
	The Admin or Department Head navigates to the "Feedback" section. 3.The Admin or Department Head reviews the feedback.	The system displays a list of feedback submitted by students and instructors. 4.The use case ends.
Alternative flow	If no feedback is available, the system displays: "No feedback available."	

Table 11:Use case Identification and Description for View feedback

2.3.1.2. Sequence Diagram

Login sequence diagram

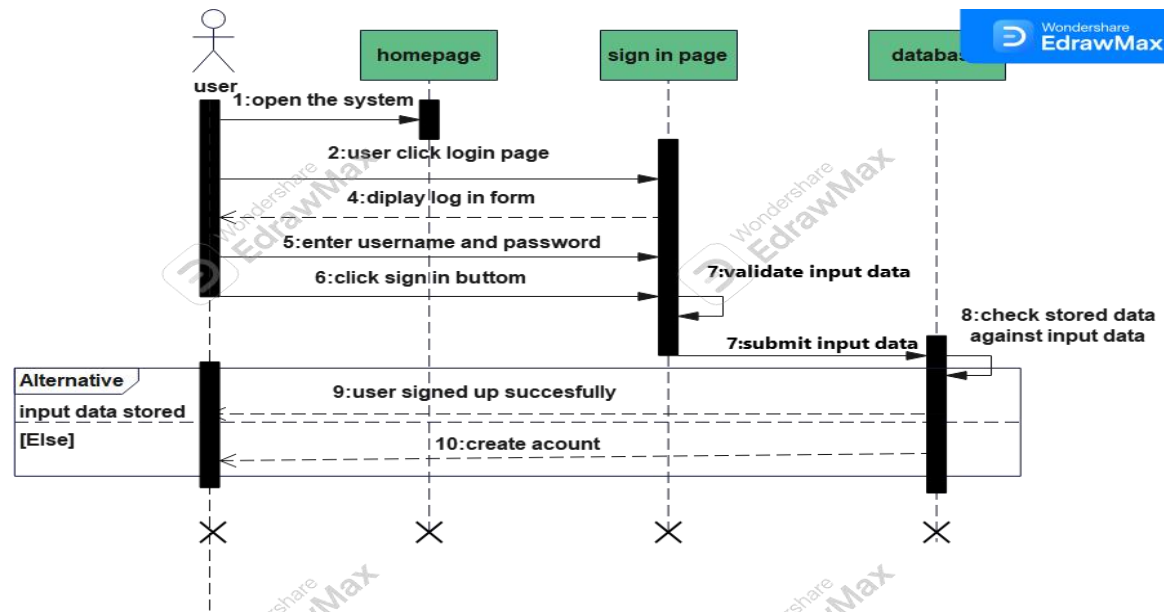


Figure 2:Login sequence diagram

Register building Login sequence diagram

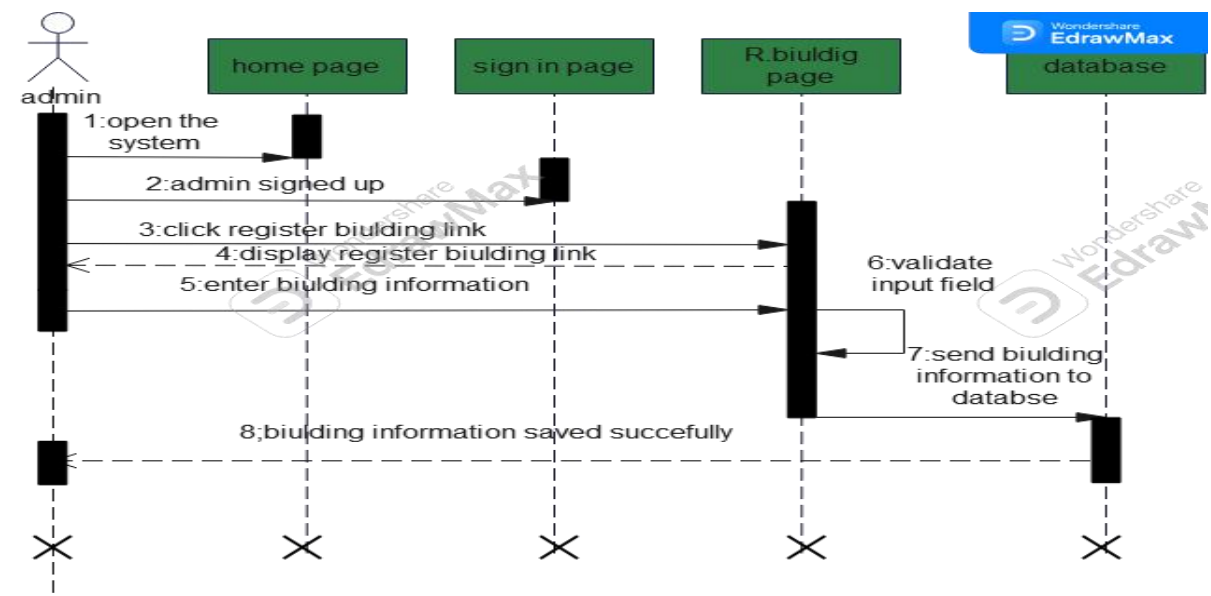


Figure 3:Register building Login sequence diagram

Submit course sequence diagram

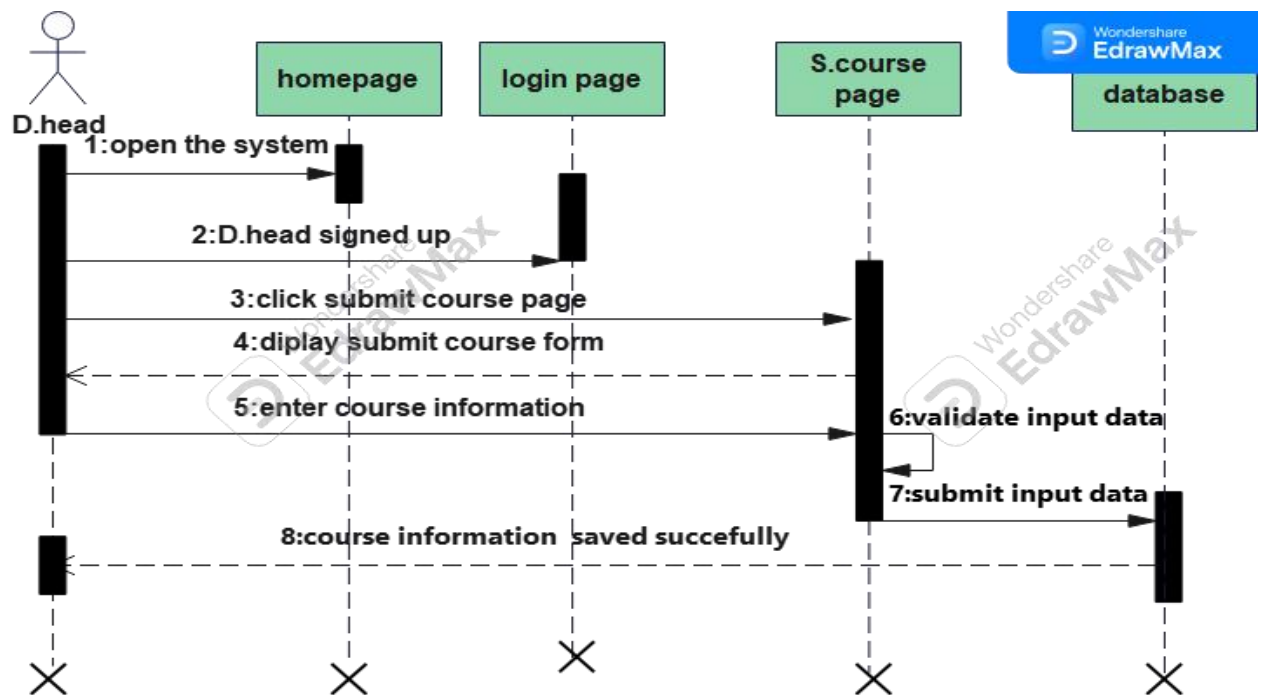


Figure 4:Submit course sequence diagram

Create class schedule sequence diagram

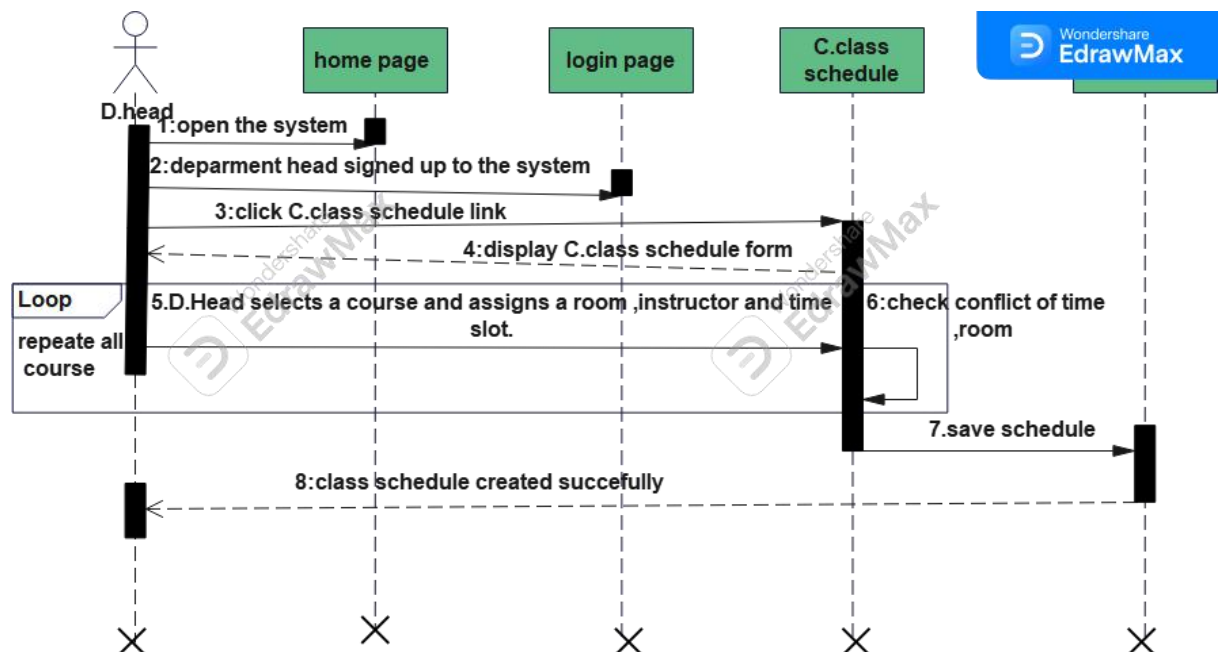


Figure 5:Create class schedule sequence diagram

Set Academic set sequence diagram

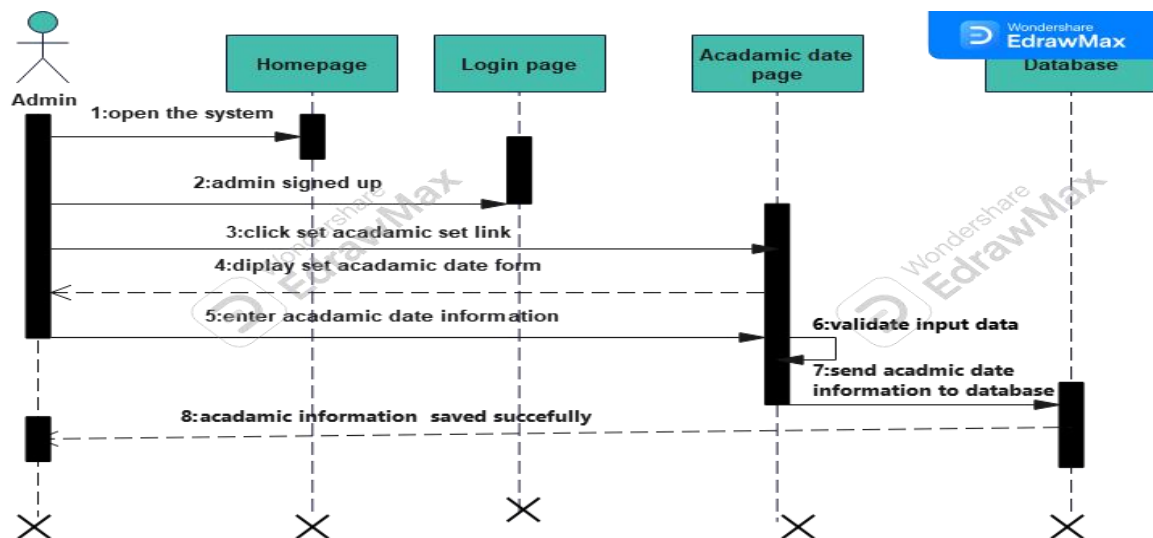


Figure 6:Set Academic set sequence diagram

Register department set sequence diagram

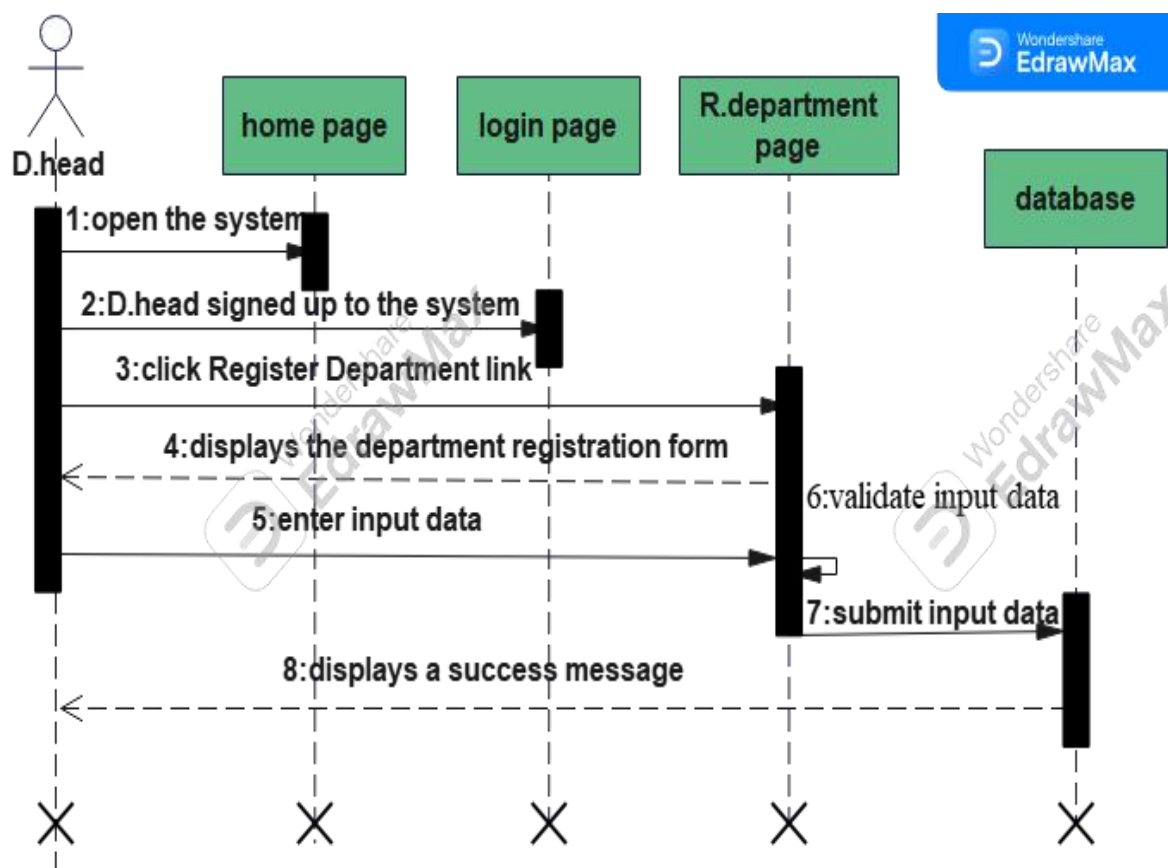


Figure 7:Register department set sequence diagram

Update class shcedule set sequence diagram

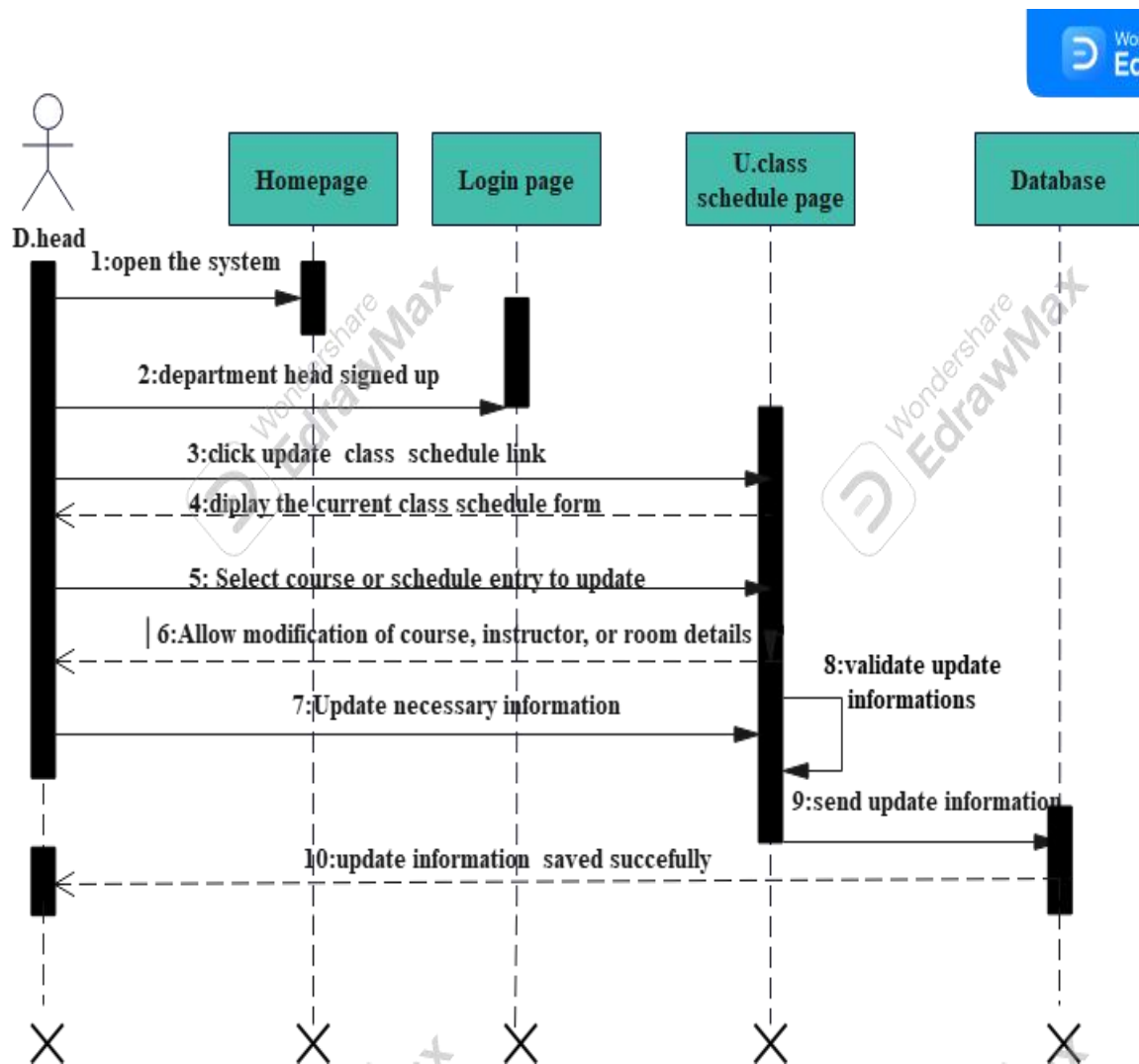


Figure 8: Update class shcedule set sequence diagram

2.3.1.3. Activity Diagram

Activity Diagram for Login

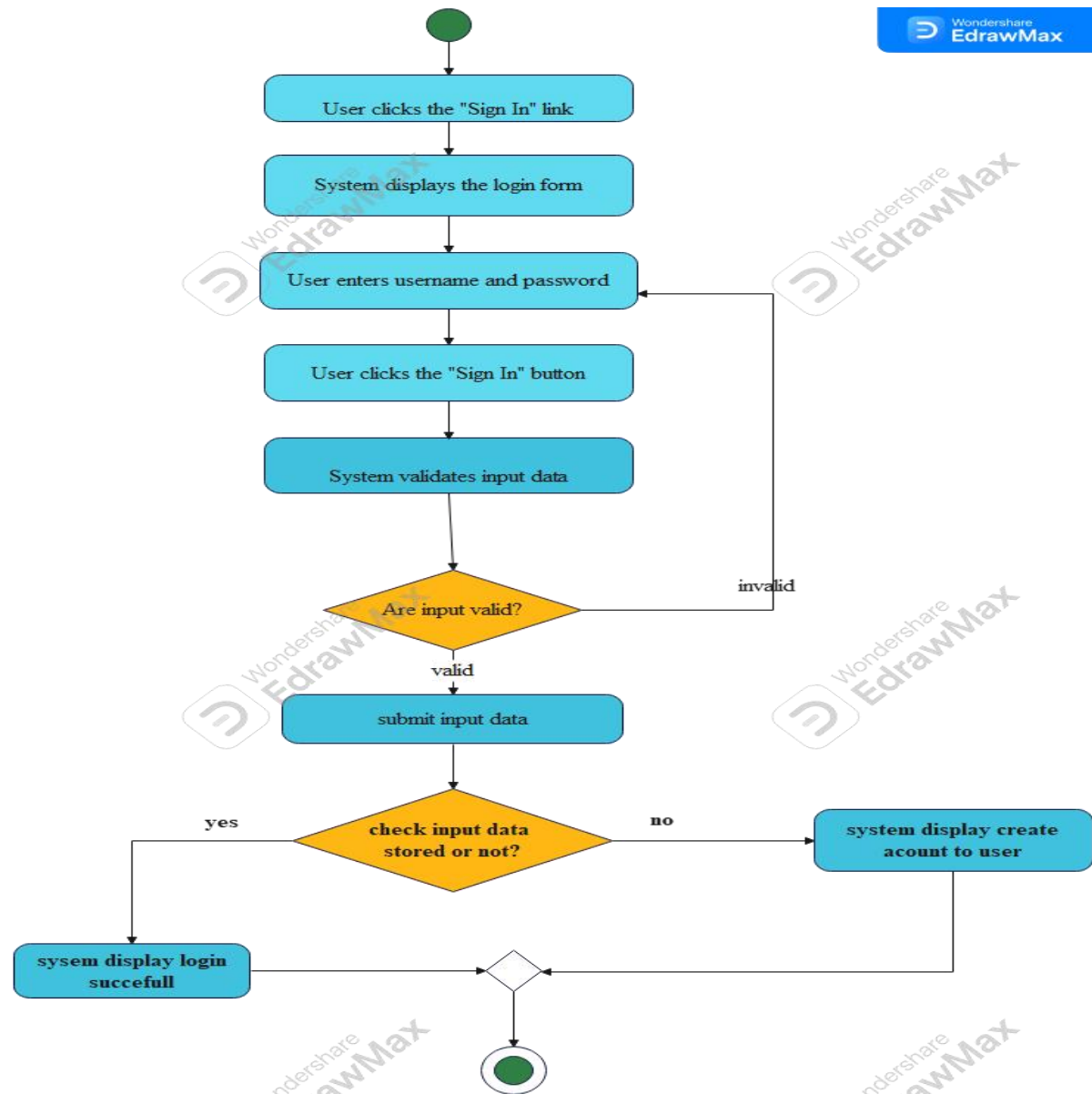


Figure 9: Activity Diagram for Login

Activity Diagram for Register Building Information

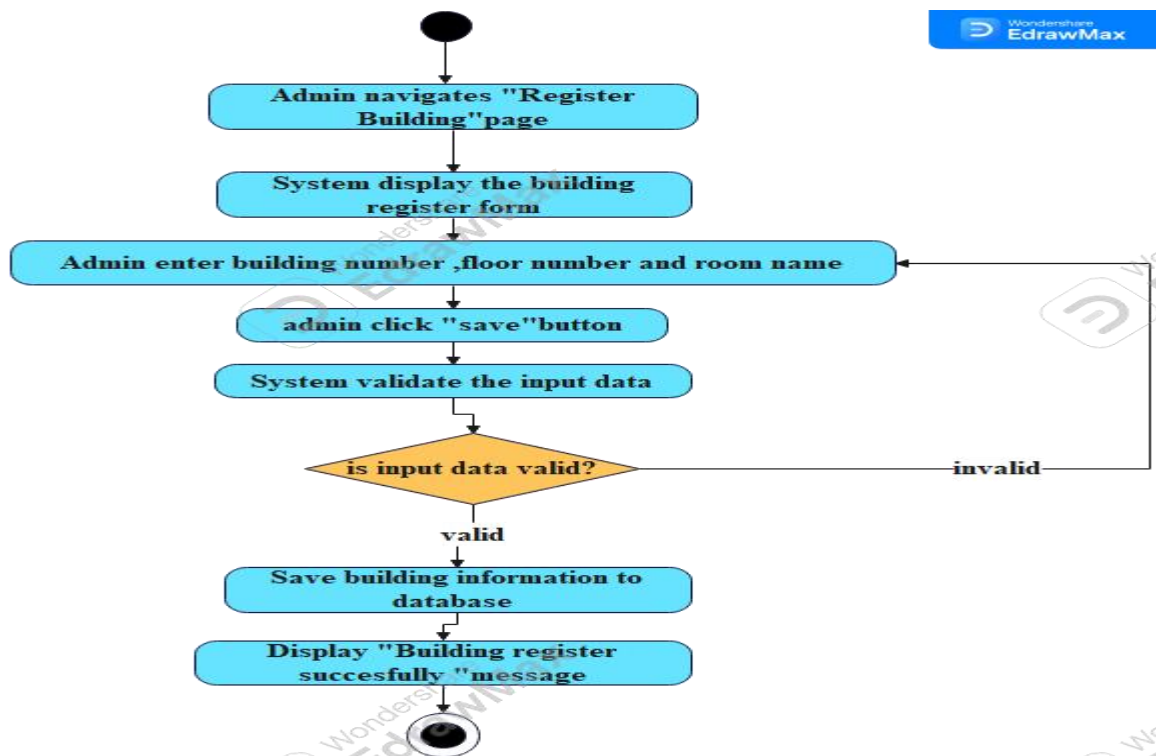


Figure 10: Activity Diagram for Register Building Information

Activity Diagram for set academic date

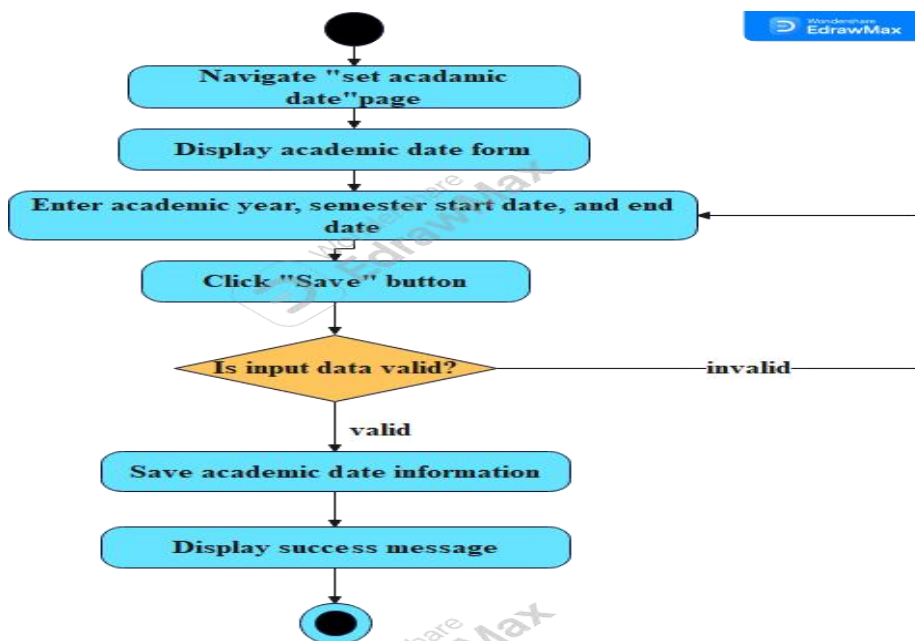


Figure 11: Activity Diagram for set academic date

Activity Diagram for create class schedule

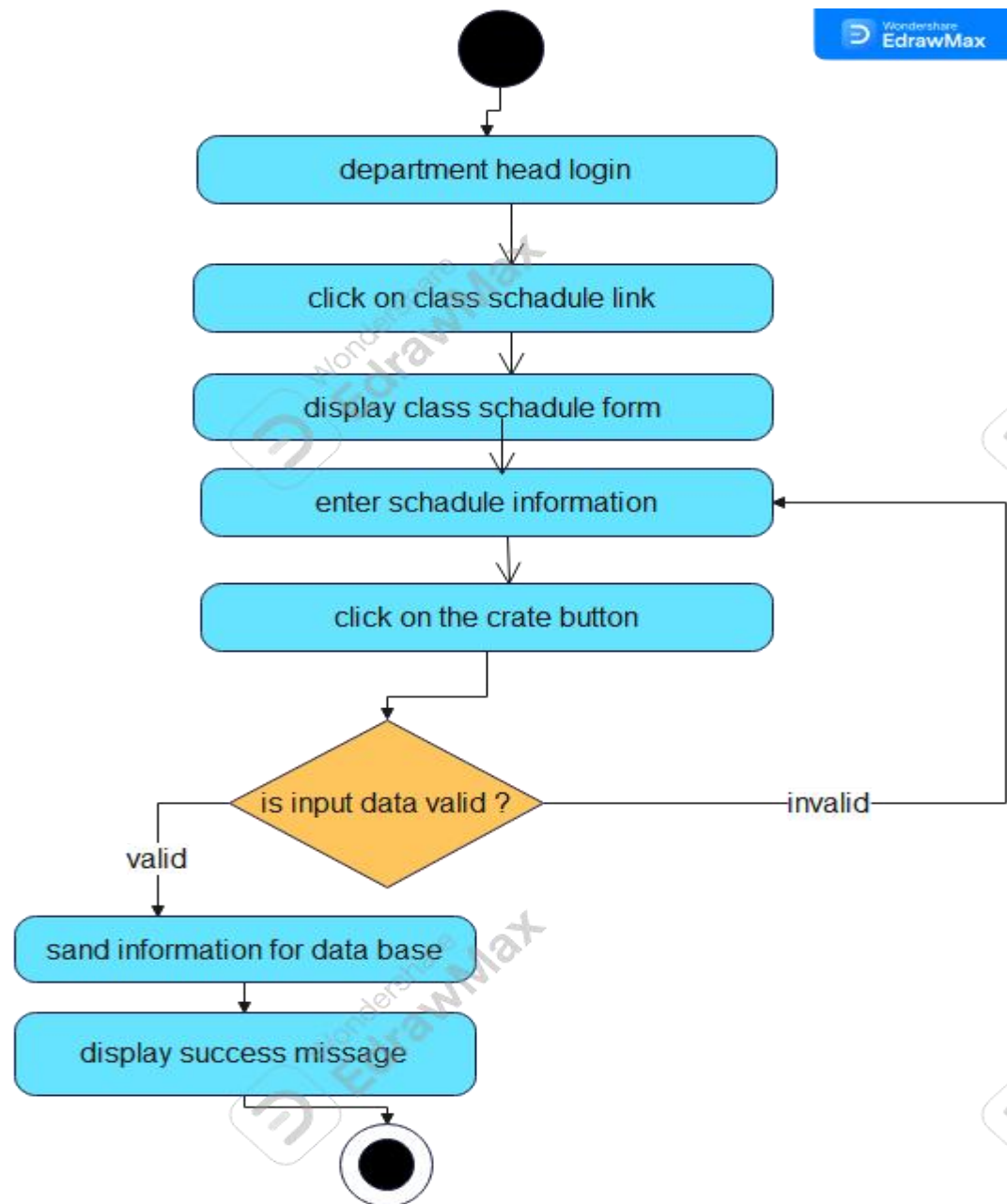
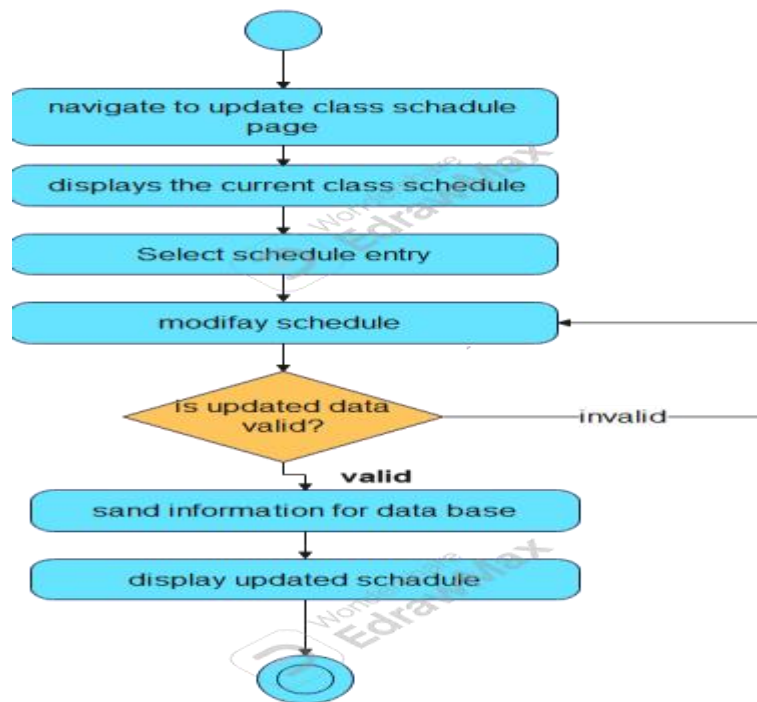


Figure 12: Activity Diagram for create class schedule

Activity Diagram for update class



sche

Figure 13: Activity Diagram for update class schedule

Activity Diagram for send feedback

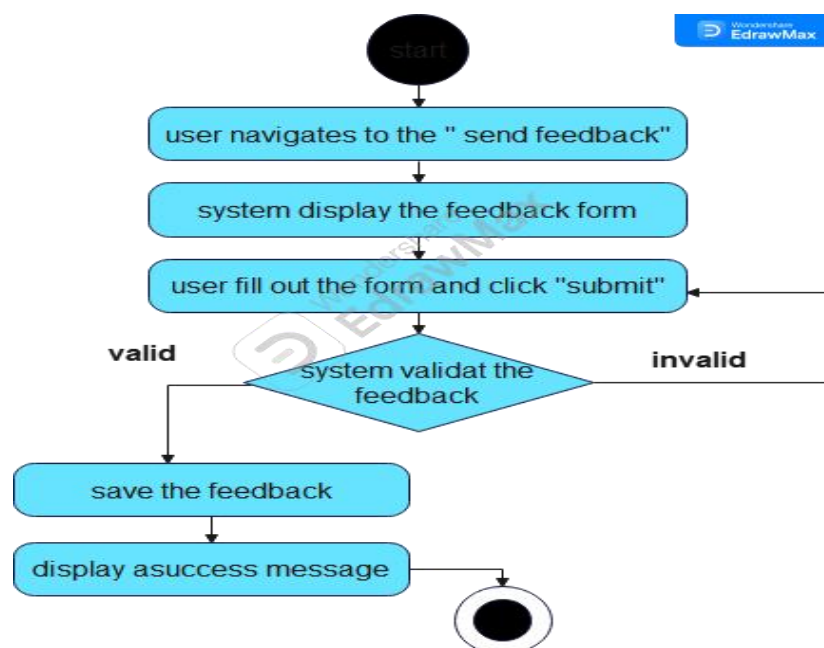


Figure 14: Activity Diagram for send feedback

Activity Diagram for Delete account

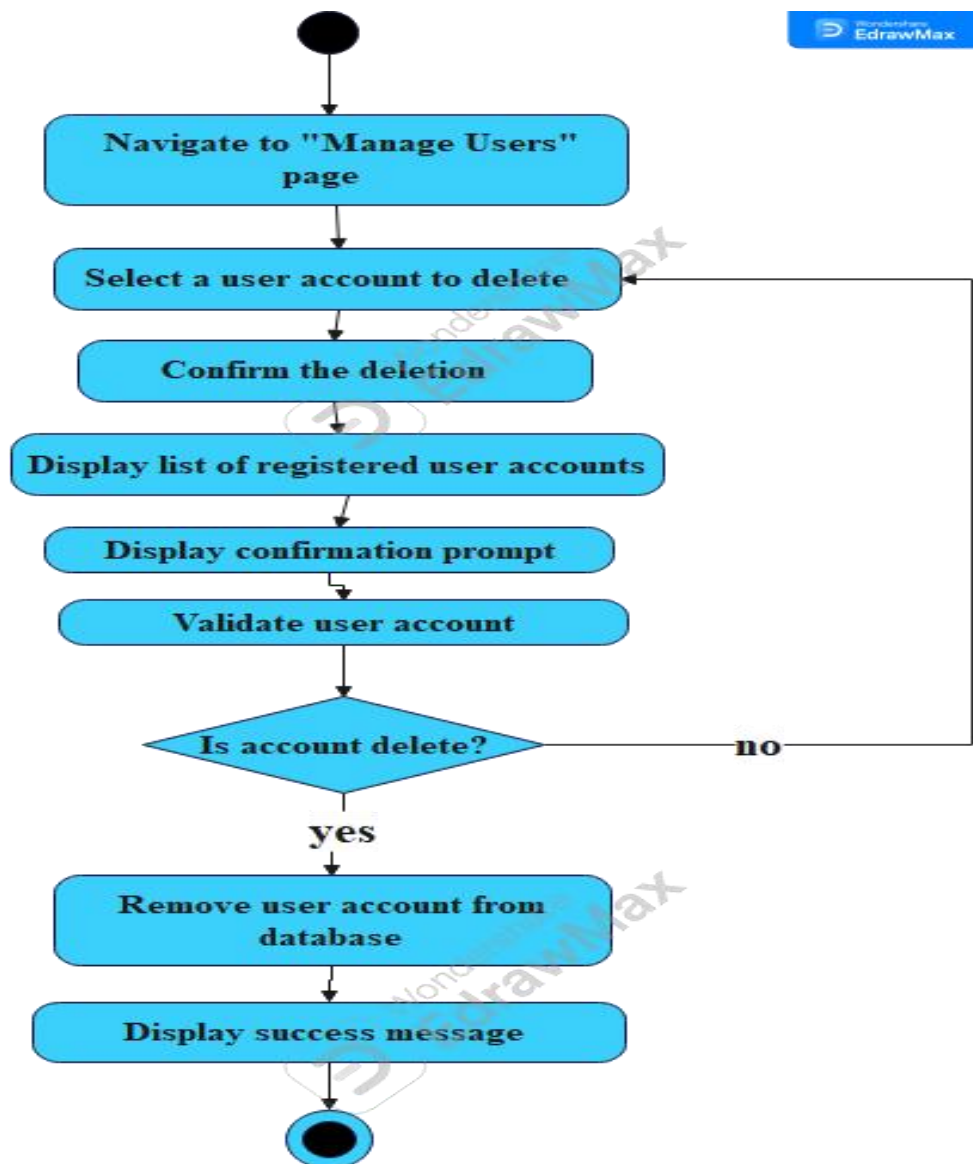


Figure 15: Activity Diagram for Delete account

2.3.1.4. Analysis Class Diagram

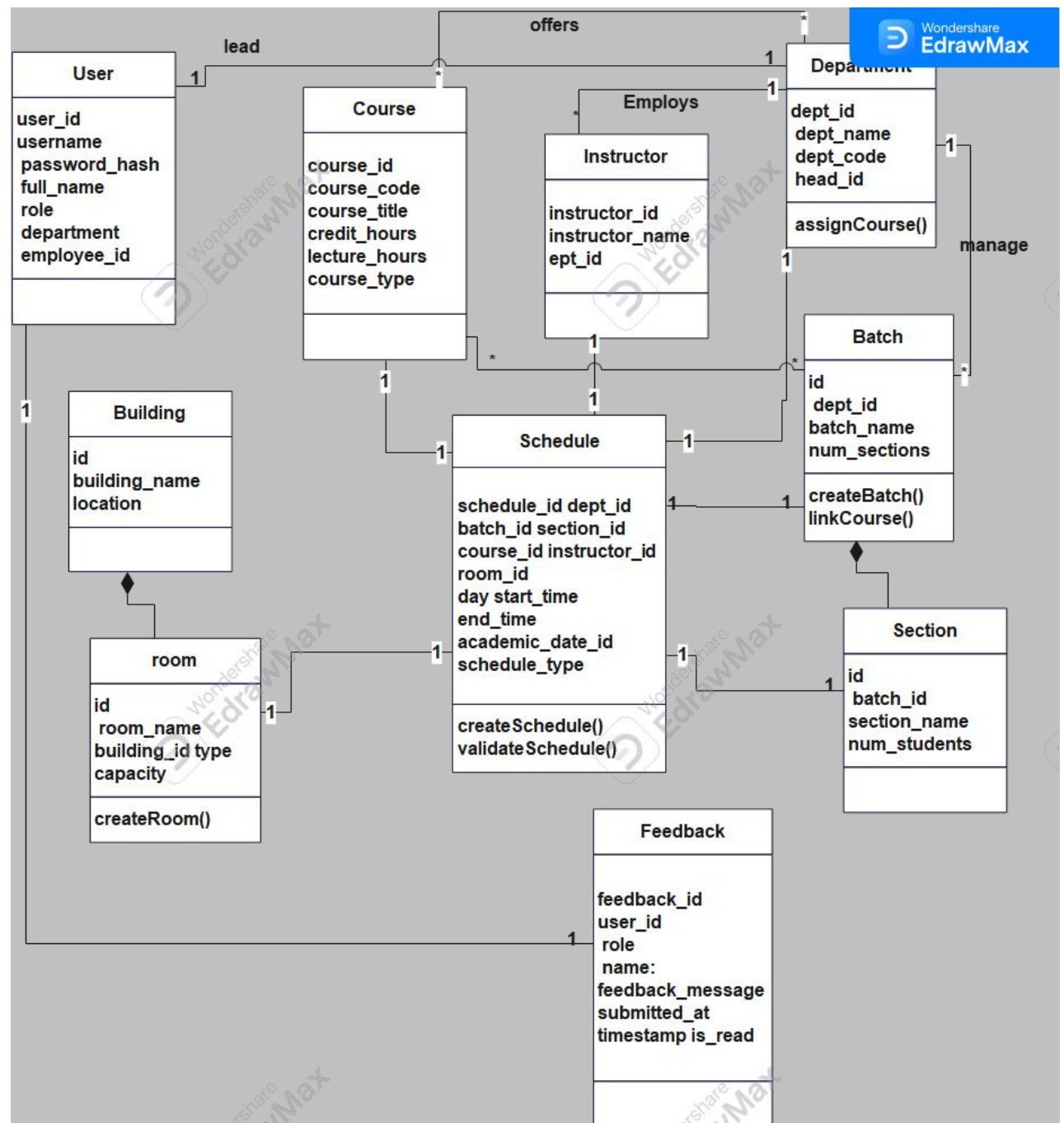


Figure 16: Analysis Class Diagram

2 Chapter three: System Design

Systems design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. The purpose of design is to determine how the system is going to build and to obtain the information needed to drive the actual implementation of the system. It focuses on understanding the model how the software will be built. System design is the detail investigation of system elements from logical view.

3.1 Design Class Diagram

Class diagram in the Unified Modelling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes:

- ✓ their attributes
- ✓ operations (or methods)
- ✓ And the relationships among the classes.

A class diagram is an illustration of the relationships and source code dependencies among classes in the Unified Modelling Language (UML).

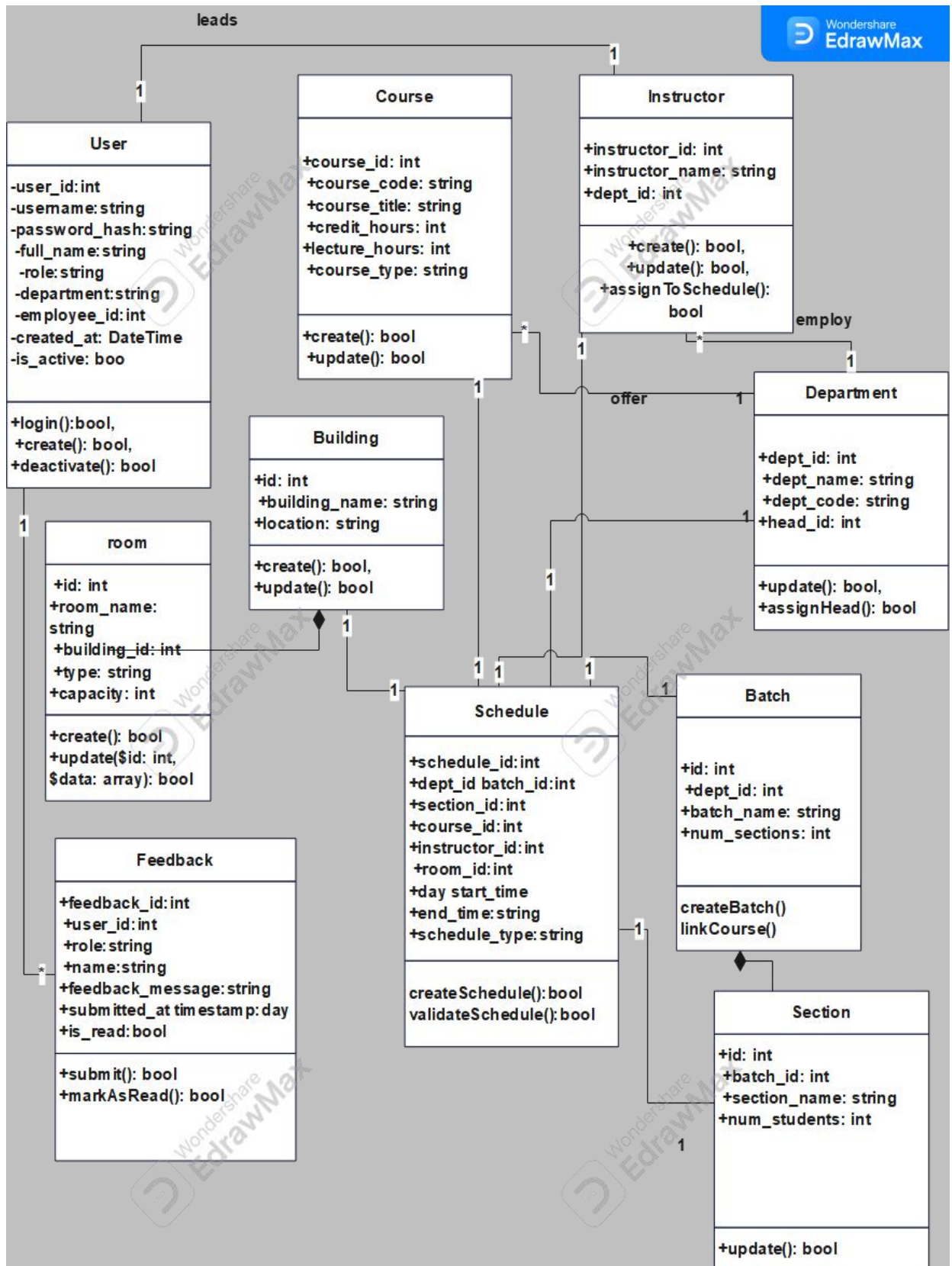


Figure 17: Design Class Diagram

3.2.Database design /Physical data model

Database design is the organization of data according to a database model. The designer determines what data must be stored and how the data elements interrelate. With this information, they can begin to fit the data to the database model. Database management system manages the data accordingly. Physical data model represents how the model built in the database. A physical database

model Shows all table structures, including column name, column data type, and column constraints.

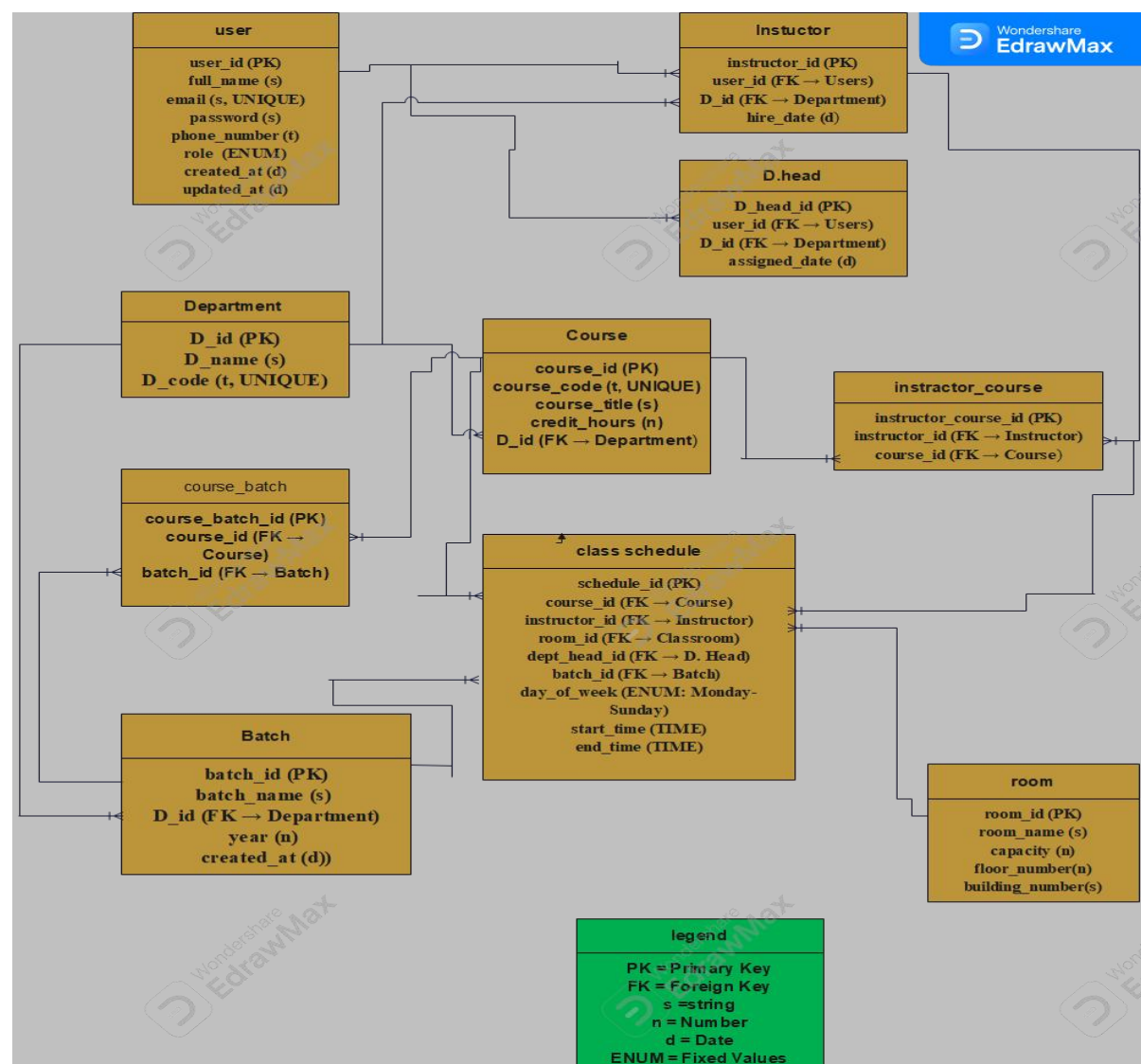


Figure 18: database modeling diagram

3.3 User Interface Design

user interface design is to make the user's interaction as simple and efficient as possible, in terms of accomplishing user goals what is often called user centred design. Good user interface design facilitates finishing the task at hand without drawing unnecessary attention to it. Graphic design may be utilized to support its usability

Home page interface

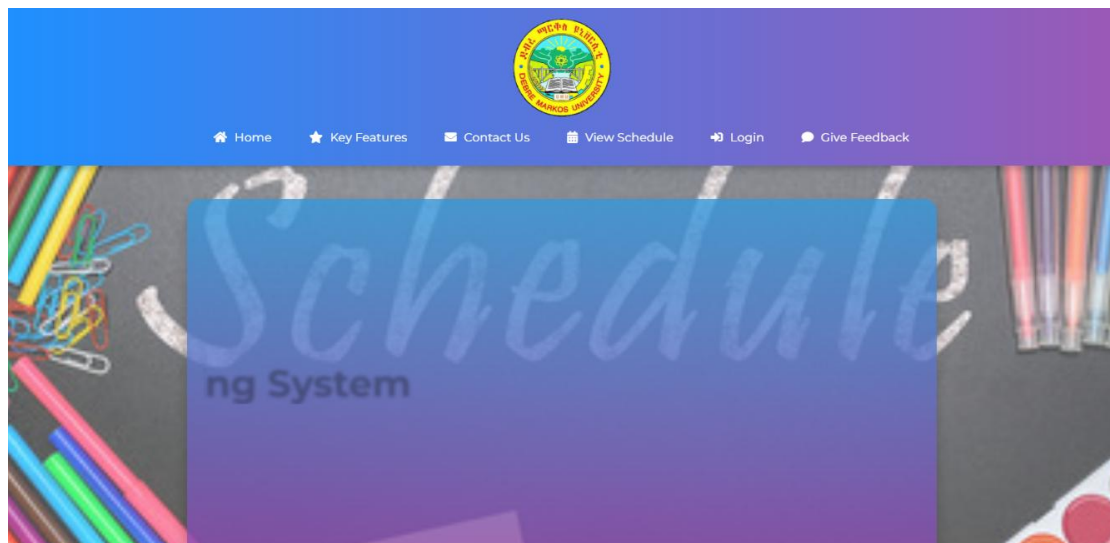


Figure 19: Home page interface

The homepage of the Debre Markos University bure campus Class Scheduling System acts as the primary entry point, offering a clear and engaging interface for all users. It features:

Navigation Menu: A sticky, gradient-styled menu with links to Key Features, Contact Us, View Schedule, Login, and Give Feedback, enabling access to system modules.

Welcome Marquee: A dynamic marquee heading, “Welcome to Class Scheduling System,” with gradient text, highlighting the system’s purpose.

Visual Design: A parallax campus background and responsive hamburger menu ensure aesthetic appeal and mobile usability .

Screenshots capture the marquee, navigation, and parallax, supporting schedule viewing , feedback , and authentication.

View schedule Interface

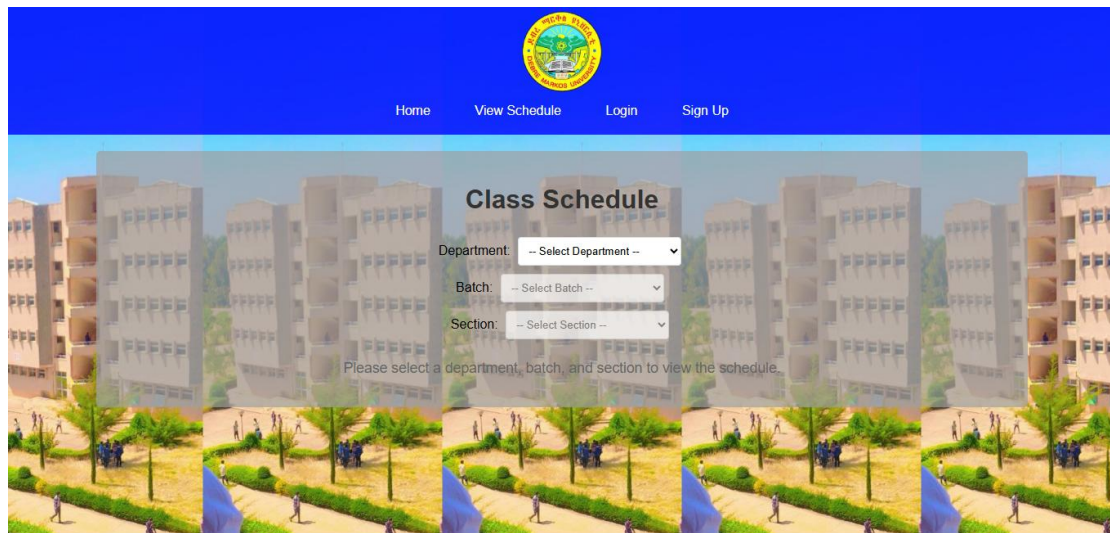


Figure 20: view schedule Interface

The View Schedule interface of the Markos University Class Scheduling System provides an intuitive platform for users to access class schedules.

It features:

Navigation Menu: A sticky, gradient-styled menu linking to Home, Key Features, Contact Us, Login, and Give Feedback, ensuring easy system navigation.

Schedule Display: A clear, tabular presentation of class schedules, detailing courses, times, and locations, accessible to all users

Visual Design: A parallax campus background and responsive layout enhance usability and aesthetics

Screenshots highlight the schedule table, navigation, and parallax,

supporting efficient schedule viewing for students, instructors, and administrators.

Admin Interface

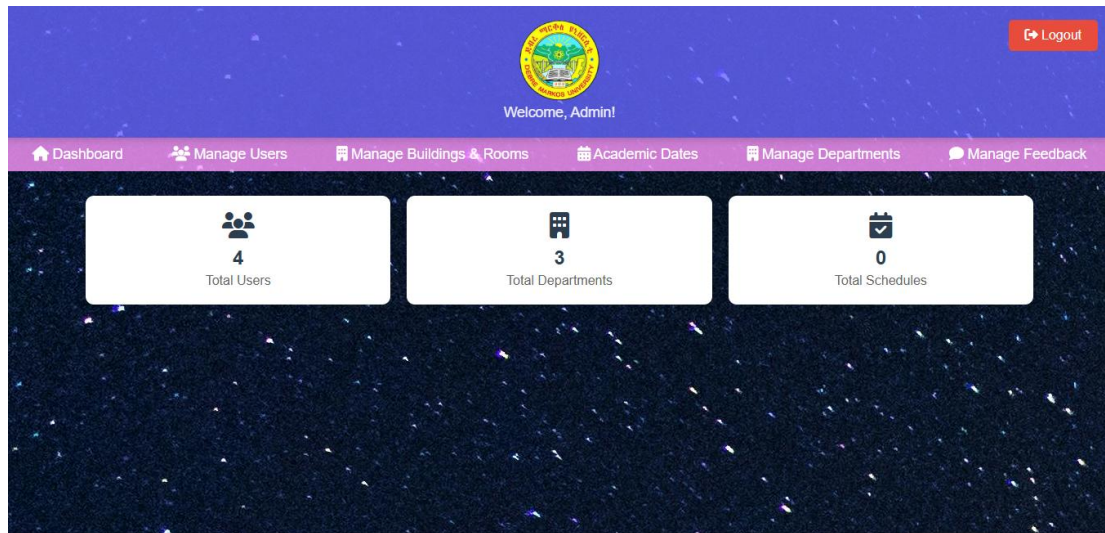


Figure 21: Admin Interface

The Admin interface provides system management functionalities, including:

Buildings and Rooms: Allows the Admin to register building information such as building number, floor number, and room names.

Set Academic Dates: Enables defining academic years, semester start and end dates.

Delete User Accounts: Provides options to remove inactive or unauthorized users.

View Feedback: Displays feedback submitted by students and instructors for administrative review.

3.4. System Architecture (Deployment Diagram)

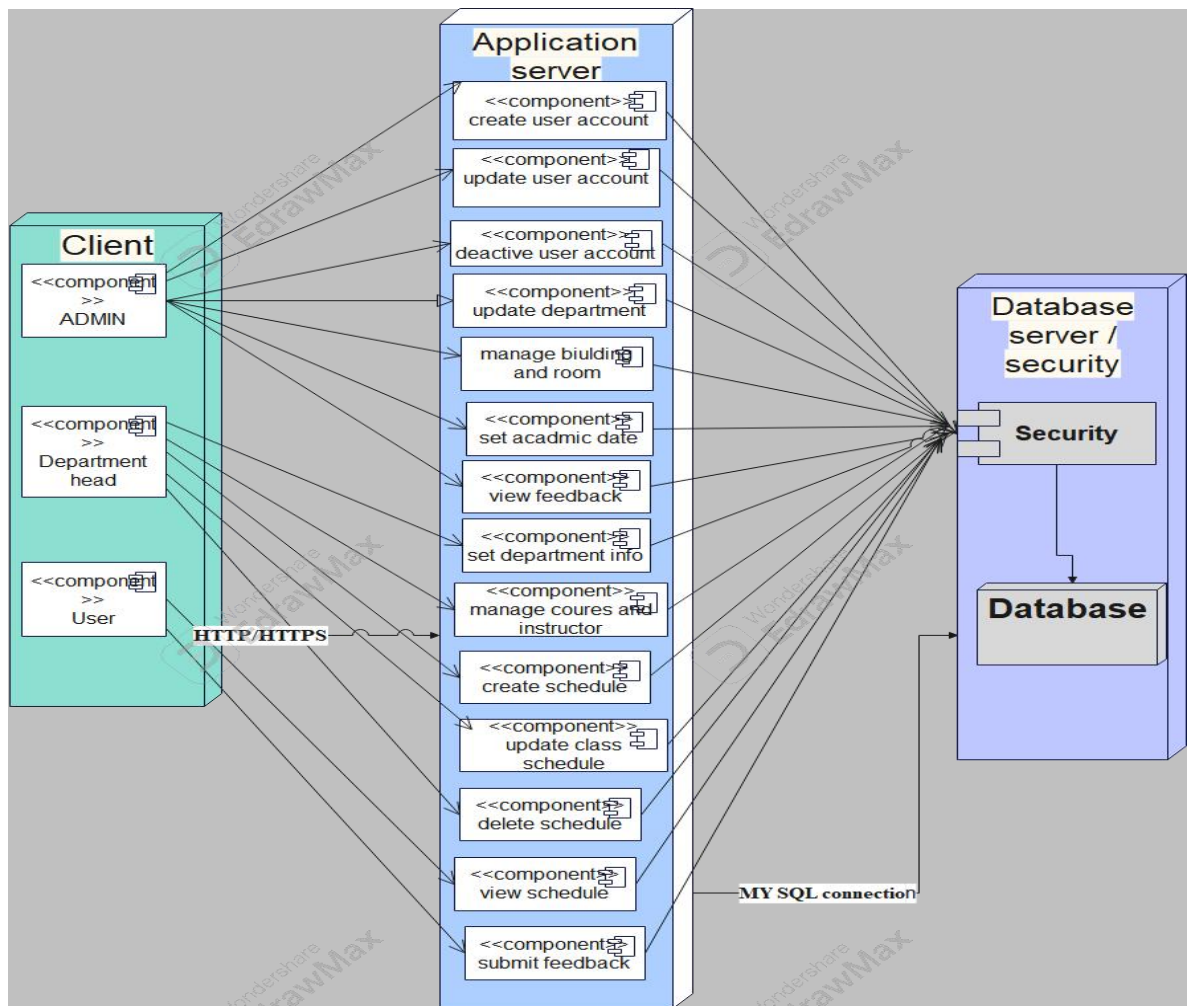


Figure 22: Deployment Diagram

3 CHAPTER 4 :IMPLEMENTATION

4.1 Overview of the programming language used

The system under development is a class scheduling management platform designed for Debre Markos University Burie Campus, to streamline the scheduling of batches, sections, courses, and instructors. It aims to enhance the efficiency of academic scheduling and resource allocation for department heads and administrators. The system architecture includes a front-end built with HTML, CSS, and JavaScript, a back-end using PHP for server-side logic, and MySQL for database management. The database, `dmu_class_scheduling`, stores critical data such as batches, sections, rooms, and schedules, with connectivity managed through `db_connection.php`.

The platform allows department heads to add, edit, and delete batches and sections, update student counts, and manage schedules, while administrators oversee courses and user roles. The back-end handles data validation, transaction management, and query execution, ensuring data integrity and security. Key features include batch and section management, course assignments, scheduling with room and instructor allocation, and user authentication for different roles.

Users include department heads, instructors, students, and administrators, with each role accessing specific functionalities based on permissions. Constraints include the need for accurate data input and the limitation of scheduling conflicts, assuming reliable internet access and a compatible web browser for operation.

4.2 Algorithms

The Debre Markos University Burie Campus Class Scheduling System utilizes the bcrypt algorithm for secure password hashing. Bcrypt, implemented via PHP's `password_hash()` function, is ideal for securely

storing user credentials in the users table, offering strong protection against brute-force attacks. Unlike simpler hashing methods, bcrypt's adaptive design and automatic salt generation make it a robust choice for user authentication in our class scheduling system.

Advantages of Bcrypt for Password Hashing:

Simplicity:

Built-in Salt: Bcrypt automatically generates and manages salts, simplifying secure password storage in `manage_user.php`.

Ease of Use: PHP's `password_hash()` and `password_verify()` functions streamline implementation in `login.php`.

Security:

Strong Cryptography: Bcrypt's computationally intensive process resists brute-force and rainbow table attacks.

Data Integrity: Ensures passwords remain secure even if the database is compromised.

Authenticity: Verifies user identity during login, preventing unauthorized access.

Key Management:

Secure Storage: Hashed passwords must be stored in a protected database to prevent unauthorized access.

No Transmission: Passwords are never sent in plaintext, ensuring secure handling during authentication.

Additional Algorithms:

Schedule Conflict Detection: In `create_schedule.php`, SQL queries check the `section_schedules` table to prevent overlapping schedules for instructors and rooms, leveraging unique constraints on `instructor_id` and `room_id`. This ensures efficient and accurate schedule creation.

Dynamic Filtering: In `public_schedule.php`, JavaScript and AJAX dynamically filter schedules by department, batch, and section, querying the `batches`, `sections`, and `section_schedules` tables to provide a responsive user experience.

These algorithms ensure secure authentication, conflict-free scheduling, and interactive schedule access, forming the core of the system's functionality.

4.3 Sample Codes

The following code from `index.php` serves as the entry point for the Debre Markos University Burie Campu Class Scheduling System, directing users to the public schedule view or login page based on their authentication status. It integrates session management and navigation, connecting to core components like `login.php` and `public_schedule.php`. For additional code examples, including user management, schedule creation, and department operations, refer to implementation files such as `manage_user.php`, `create_schedule.php`, and `submit_dept_info.php` in the project repository. These files illustrate the system's comprehensive functionality and database interactions.

```

<?php
// Enable error reporting for debugging
ini_set('display_errors', 1);
error_reporting(E_ALL);
?>
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Home - Class Scheduling</title>
  <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.15.4/css/all.min.css">
  <style>
    body {
      font-family: 'Poppins', Arial, sans-serif;
      margin: 0;
      padding: 0;
      background-image: url('image/yhalem.jpg');
      background-size: cover;
      background-position: center;
      background-repeat: no-repeat;
      background-attachment: fixed; /* Ensure full coverage even on scroll */
      min-height: 100vh;
      height: 100%; /* Ensure the body fills the viewport */
    }
    .header {
      background: linear-gradient(to right, #3498db, #8e44ad);
      box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
      padding: 20px 0;
      text-align: center;
    }
    .logo {
      max-width: 150px;
      transition: transform 0.3s ease;
    }
    .nav ul {
      display: flex;
      justify-content: center;
      gap: 20px;
      margin-top: 10px;
      list-style: none;
      padding: 0;
      margin: 0;
    }
    .nav-link {
      color: white;
      text-decoration: none;
      padding: 12px 18px;
      border-radius: 5px;
      transition: background-color 0.3s;
    }
    .nav-link:hover {
      background-color: rgba(255, 255, 255, 0.2);
    }
    .nav-link i {
      margin-right: 5px;
    }
    .main {
      background-color: rgba(255, 255, 255, 0.85);
      padding: 40px;
      margin: 40px auto;
      width: 90%;
      max-width: 800px;
      border-radius: 10px;
      box-shadow: 0 6px 10px rgba(0, 0, 0, 0.1);
      animation: fadeIn 1s ease;
      text-align: center;
    }
    .heading {
      font-size: 3em;
      color: #2c3e50;
      text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.1);
      margin-bottom: 20px;
    }
    .tagline {
      font-size: 1.5em;
      color: #7f8c8d;
      margin-bottom: 30px;
    }
    .cta-button {

```

```

.cta-button {
  background-color: #c74c3c;
  color: white;
  padding: 15px 30px;
  border-radius: 30px;
  text-decoration: none;
  font-weight: bold;
  transition: background-color 0.3s, box-shadow 0.3s;
}
.cta-button:hover {
  background-color: #e0392b;
  box-shadow: 0 0 15px #e74c3c;
}
.section-heading {
  font-size: 2em;
  color: #2c3e50;
  margin: 30px 0 15px;
}
.section-text {
  font-size: 1.1em;
  color: #34495e;
  line-height: 1.6;
  margin-bottom: 20px;
}
.about-us {
  border: 2px solid #3498db;
  border-radius: 10px;
  padding: 20px;
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
  background-color: rgba(255, 255, 255, 0.9);
  margin-bottom: 20px;
}
.key-features {
  border: 2px solid #3498db;
  border-radius: 10px;
  padding: 20px;
  margin-bottom: 20px;
}
padding: 20px;
box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
background-color: rgba(255, 255, 255, 0.9);
margin-bottom: 20px;
}
.key-features ul {
  list-style-type: disc;
  padding-left: 20px;
  text-align: left;
  color: #34495e;
}
.key-features li {
  margin-bottom: 10px;
  font-size: 1.1em;
}
.contact-info {
  font-size: 1em;
  color: #34495e;
  margin-bottom: 10px;
}
.contact-info 1 {
  margin-right: 40px;
  color: #e74c3c;
}
.footer {
  background-color: #34495e;
  color: white;
  padding: 20px 0;
  text-align: center;
}
.social-links a {
  color: white;
  margin: 0 10px;
  font-size: 1.1em;
  transition: color 0.3s;
}

```

```

)
padding: 15px;
}
</style>
</head>
<body>
<header class="header">

<nav class="nav">
<ul>
<li><a class="nav-link" href="index.php"><i class="fas fa-home"></i> Home</a></li>
<li><a class="nav-link" href="public_schedule.php"><i class="fas fa-calendar-alt"></i> View Schedule</a></li>
<li><a class="nav-link" href="login.php" target="_self"><i class="fas fa-sign-in-alt"></i> Login</a></li>
<li><a class="nav-link" href="signup.php"><i class="fas fa-user-plus"></i> Sign Up</a></li>
<li><a class="nav-link" href="feedback.php"><i class="fas fa-comment"></i> Give Feedback</a></li>
</ul>
</nav>
</header>
<main class="main">
<h1 class="heading"><marquee>Welcome to Class Scheduling System</marquee></h1>

<!-- About Us Section with Border -->
<div class="about-us">
<h2 class="section-heading">About Us</h2>
<p class="section-text">
The Class Scheduling System at Markos University is designed to streamline the process of managing academic schedules for students, faculty, and administrators. Our mission is to provide an intuitive platform that ensures efficient scheduling, reduces conflicts, and enhances the overall educational experience. We are committed to supporting the university community with innovative tools and reliable services.
</p>
</div>

<!-- Key Features Section -->
<div class="key-features">
<h2 class="section-heading">Key Features</h2>
<ul>

```

```

<!-- Key Features Section -->
<div class="key-features">
  <h2 class="section-heading">Key Features</h2>
  <ul>
    <li>Real-time schedule updates for students and faculty.</li>
    <li>Automatic conflict detection and resolution.</li>
    <li>User-friendly interface for easy navigation.</li>
    <li>Administrative tools for efficient schedule planning.</li>
  </ul>
</div>

<!-- Contact Us Section -->
<h2 class="section-heading">Contact Us</h2>
<p class="contact-info"><i class="fas fa-envelope"></i> Email: dessieaweke@gmail.com</p>
<p class="contact-info"><i class="fas fa-envelope"></i> Email: yhalemayalu@gmail.com</p>
<p class="contact-info"><i class="fas fa-map-marker-alt"></i> Address: Debre Markos University, Burie Campus, Burie Town, Ethiopia</p>
</main>
<footer class="footer">
  <p>© 2025 Markos University. All rights reserved. This site and its content are protected under international copyright laws.</p>
  <div class="social-links">
    <a href="#"><i class="fab fa-facebook"></i></a>
    <a href="#"><i class="fab fa-twitter"></i></a>
    <a href="#"><i class="fab fa-instagram"></i></a>
  </div>
</footer>
</body>
</html>

```

4 CHAPTER 5: TESTING

5.1 Sample Test

The Debre Markos University Burie Campus Class Scheduling System was thoroughly tested to ensure its functionality, reliability, and usability for all user roles, including admins, department heads, and public users. The testing process followed a structured approach, with sample test cases designed to validate critical system components. The steps below outline the testing methodology and specific test cases executed to confirm the system's performance at Debre Markos University Burie Campus.

Test Environment Setup: Testing was conducted in a controlled environment mirroring the production setup, using XAMPP with PHP 8.0 and . The database was initialized with the `dmu_class_scheduling.sql` schema, populating tables like `users`, `departments`, and `section_schedules` with sample data (e.g., admin user “dessie,” department “computer science”). This ensured consistent and reliable test results.

User Authentication Test (login.php): A test case verified the login functionality for an admin user. The input was a valid username (“dessie”) and password (matching the bcrypt-hashed value in the `users` table). The

expected outcome was a successful redirect to `admin_dashboard.php` with session variables set (e.g., `$_SESSION['user_id'] = 5`). An additional test with invalid credentials (e.g., incorrect password) confirmed that an error message (“Invalid username or password”) was displayed, and no session was created. This validated secure authentication and role-based access control.

Schedule Creation Test (`create_schedule.php`): A test case checked the creation of a new schedule for the Computer Science department (`dept_id`: 7, `batch_id`: 1, `section_id`: 1). Inputs included a course (e.g., “cs101”), instructor (e.g., “robel M”), room (e.g., “103”), day (“Monday”), and time (“08:00-10:00”). The system queried the `section_schedules` table to ensure no conflicts existed for the instructor or room at the specified time. The expected outcome was a new entry in `section_schedules` with correct details and a success message (“Schedule created successfully”). A conflicting schedule test (same instructor, same time) was rejected, confirming the conflict detection algorithm’s effectiveness.

Public Schedule Viewing Test (`public_schedule.php`): A test case validated the dynamic schedule filtering feature. The user selected a department (“computer science”), batch, and section via dropdowns. JavaScript and AJAX calls fetched corresponding data from the `batches`, `sections`, and `section_schedules` tables, populating a table with course details (e.g., course code, instructor, room, time). The expected outcome was an accurate, formatted schedule display, with print-friendly styling applied via CSS media queries. A test with an empty selection (e.g., no department chosen) ensured no data was displayed, verifying input validation.

Feedback Submission Test (`submit_feedback.php`): A test case simulated a student submitting feedback. Inputs included role (“Student”), name

("kidu"), and feedback message ("System is user-friendly"). The system inserted the data into the feedback table with `is_read = 0` and the current timestamp. The expected outcome was a success message ("Feedback submitted successfully!") and a new entry in the feedback table (e.g., `feedback_id: 6`). A test with missing fields (e.g., empty message) was rejected, confirming client-side and server-side validation.

These sample tests, executed manually in the XAMPP environment, validated the system's core functionalities, including authentication, schedule management, and user interaction. The results confirmed seamless integration between the PHP front-end, MySQL database, and JavaScript-driven interface, ensuring the system met the operational needs of Debre Markos University Burie Campus. For further details on test cases and results, refer to the project's testing logs in the repository

5 CHAPTER 6: Conclusion and Recommendation

6.1. Conclusion

The Debre Markos University bure campus Class Scheduling System successfully addresses the challenges of manual scheduling at Debre Markos University Burie Campus by providing an efficient, user-friendly, and secure web-based platform. Developed using PHP, MySQL, HTML, CSS, and JavaScript, the system streamlines schedule creation, management, and access for admins, department heads, and public users. Key features, such as role-based authentication (`login.php`), conflict-free schedule generation (`create_schedule.php`), dynamic schedule viewing (`public_schedule.php`), and feedback collection (`submit_feedback.php`), were implemented and rigorously tested to ensure reliability and usability. The MySQL database, with 13 tables like `users`, `departments`, and `section_schedules`, supports robust data management, while security measures like bcrypt password hashing and input sanitization protect user

data. Testing confirmed seamless integration of all components, meeting the project's objectives of automating scheduling, enhancing accessibility, and improving operational efficiency. The system delivers a scalable solution that significantly reduces administrative workload and enhances the scheduling experience for the university community.

6.2. Recommendation

While the Debre Markos University Class Scheduling System effectively automates scheduling at Debre Markos University Burie Campus, several enhancements can further improve its functionality and user experience. It is recommended to integrate automated conflict resolution algorithms in `create_schedule.php` to suggest alternative time slots or rooms when conflicts arise, enhancing efficiency for department heads. Adding a mobile application, built with frameworks like React Native, would improve accessibility for students and staff, extending the functionality of `public_schedule.php` to mobile devices. Implementing email notifications for schedule updates or feedback responses, linked to `submit_feedback.php` and `view_schedule.php`, would enhance communication. Strengthening security by adopting prepared statements in all database queries across files like `manage_user.php` and `submit_course_instructor.php` would further protect against SQL injection risks. Finally, introducing a reporting module to generate analytics on room utilization or instructor workloads, leveraging data from the `section_schedules` and `rooms` tables, would support administrative decision-making. These enhancements would make the system more robust, user-friendly, and adaptable to the university's evolving needs.

6.3. Future enhancement

The Markos University Class Scheduling System provides a solid foundation for automating scheduling at Debre Markos University Burie Campus, but future enhancements can significantly elevate its capabilities. Integrating machine learning algorithms in `create_schedule.php` to predict optimal schedules based on historical data from the `section_schedules` table could minimize conflicts and optimize resource allocation. Developing a dedicated mobile application using Flutter would enhance accessibility, allowing students to access `public_schedule.php` features and receive push notifications for schedule changes. Adding a real-time chat feature, integrated with `submit_feedback.php`, would enable direct communication between users and admins for immediate query resolution. Upgrading the database to support advanced analytics, such as predictive enrollment trends using the `batches` and `courses` tables, would aid strategic planning. Finally, implementing single sign-on (SSO) authentication in `login.php` to integrate with university systems would streamline user access. These enhancements would ensure the system remains scalable, user-centric, and aligned with the university's long-term operational goals.

6 References

- ❖ Anderson, J.A. and Smith, L. (2015) 'Automated Scheduling Algorithms in Web-based Systems', *Journal of Computer Science*, Vol. 38, pp. 112-123.
- ❖ own, T.D. (2019) 'Web Development Frameworks: A Comparative Study', *Journal of Web Technologies*, Vol. 12, pp. 201-210.
- ❖ enkins, H.M. and Wright, R.S. (2018) 'SQL and Database Management for Online Systems', *International Journal of Database Design*, Vol. 45, pp. 77-85.
- ❖ iller, G.E. (2017) 'PHP for Beginners: Building Dynamic Websites', *Web Development Press*, pp. 35-50.
- ❖ hao, L. and Zhang, Q. (2020) 'A Review of Scheduling Systems for Educational Institutions', *Educational Technology Journal*, Vol. 25, pp. 34-47